Quaderno di lavoro

Come funziona, e quanto costa

Dal modello al datapath reale

misure su una popolazione di 109 modelli

Progetto **IPA/eBPF**

inferenza di una rete neurale dentro il kernel, a bordo del pacchetto

Ogni numero di questo quaderno viene da una misura eseguita, non da una stima. Dove una cosa non è stata verificata, è scritto.

Indice

1 Di che cosa stiamo parlando **2**
1.1 L'idea in una pagina . . . 2
1.2 Dove gira: XDP . . . 2
1.3 Le tre pipeline . . . 2

2 La matematica che sta dentro il kernel **4**
2.1 Perché i numeri sono interi . . . 4
2.2 I bias vanno riportati nelle unità dell'accumulatore . . . 4
2.3 Il TTL: dividere il prodotto, mai il valore . . . 5

3 Le classi non sono porte **6**
3.1 Che cosa significa una classe . . . 6
3.2 Una catena esplicita . . . 6
3.3 Ogni modello ha la sua semantica . . . 6

4 Il motore non è lo scenario **8**
4.1 Le dimensioni vengono dalla configurazione . . . 8
4.2 La prova . . . 8

5 Il modello non è quello depositato **10**
5.1 La domanda . . . 10
5.2 I modelli sintetici . . . 10
5.3 Il confronto a quattro vie . . . 10
5.4 Il risultato . . . 11

6 Dalla chiamata di sistema al kernel vero **13**
6.1 Due banchi . . . 13
6.2 Come il test evita di essere vuoto . . . 13
6.3 Il risultato . . . 14

7 Gli ingressi che dipendono dal nodo **15**
7.1 L'interfaccia d'ingresso: una porta logica . . . 15
7.2 L'identità del nodo . . . 15
7.3 La prova che contano . . . 15

8 Il verificatore paga i cammini **17**
8.1 Che cosa fa il verificatore . . . 17
8.2 Il primo strato: feature fuori, neuroni dentro . . . 17
8.3 La ReLU: un bivio per neurone . . . 17
8.4 La ReLU senza bivio: il risultato . . . 19
8.5 La ReLU senza bivio: che cosa costa . . . 19

9 Il test reale: quanto costa un pacchetto al nodo **21**
9.1 La domanda, in parole semplici . . . 21
9.2 Com'è fatto il banco . . . 21
9.3 Come si misura, e perché così . . . 22
9.4 Il generatore . . . 22
9.5 Perché un core vero, tutto suo . . . 23
9.6 I limiti del test, con esempi . . . 24
9.7 Il risultato: quanto regge il nodo . . . 25
9.8 Uno scrittore per coda . . . 26
9.9 Al crescere del bit rate . . . 27
9.10 Due core . . . 28
9.11 Il nodo sui core lenti . . . 28
9.12 Che cosa questo numero non dice . . . 29

10 La macchina di laboratorio, sotto controllo **30**
10.1 Un processore che non sta fermo . . . 30
10.2 Le condizioni, messe e tolte . . . 30
10.3 La frequenza scritta non è la frequenza vera . . . 31
10.4 Quale frequenza . . . 31
10.5 Il risultato: misure che si ripetono . . . 31
10.6 Come si rifà tutto . . . 32

11 Altri modelli, altre reti **33**
11.1 La domanda . . . 33
11.2 Scenario e modello . . . 33
11.3 Quando un modello può girare su una rete . . . 33
11.4 Come è costruito . . . 34
11.5 Che cosa controllano i test . . . 34
11.6 La prova che la strada generica non cambia niente . . . 35
11.7 Che cosa non fa . . . 35

12 Nessun modello privilegiato: la popolazione **36**
12.1 La domanda . . . 36
12.2 La popolazione . . . 36
12.3 Come si misura, e quanto è pulito il dato . . . 37
12.4 Correttezza . . . 38
12.5 Il costo del nodo . . . 38
12.6 P-core, E-core, LP E-core . . . 38
12.7 Sotto traffico crescente, e con due core . . . 39
12.8 Il programma, il trasporto, e due modi di misurare la rete . . . 39
12.9 La legge di costo . . . 40
12.10 La forma del modello . . . 41
12.11 Il costo di uno strato, a moltiplicazioni ferme . . . 42
12.12 Il programma da solo, e il cambio di modello . . . 43
12.13 Il resto è solido . . . 44
12.14 Le affermazioni, e su che cosa reggono . . . 44
12.15 Che cosa non dice . . . 45
12.16 Come rifarla . . . 45

13 Cambiare modello senza fermare i pacchetti **46**
13.1 La domanda . . . 46
13.2 Le tabelle di un modello . . . 46
13.3 Il problema . . . 46
13.4 La soluzione . . . 47
13.5 La prova . . . 47
13.6 Quanto costa installare un modello . . . 48
13.7 Che cosa non dice . . . 48
13.8 Come rifarla . . . 48

14 I numeri in una pagina **49**

15 Quel che resta aperto **50**

16 I comandi **51**
16.1 La macchina . . . 51
16.2 Correttezza: il modello fa quello che deve . . . 51
16.3 Kernel: si carica, e quanto costa . . . 51
16.4 Altri modelli, altre reti . . . 52
16.5 Datapath reale: il pacchetto attraversa davvero . . . 52
16.6 Analisi parametrica: come cresce il costo . . . 52
16.7 Il test reale . . . 52
16.8 Costi specifici e oggetto precompilato . . . 53
16.9 L'ordine consigliato . . . 53

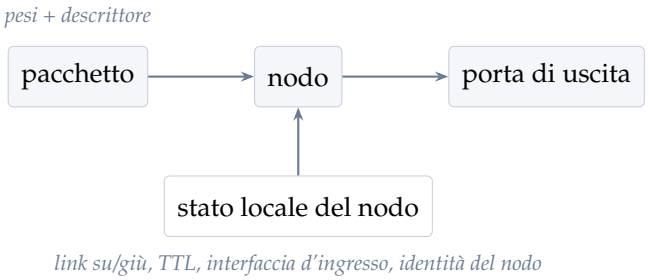
CAPITOLO 1

Di che cosa stiamo parlando

1.1 L'idea in una pagina

Un router normale decide dove mandare un pacchetto consultando una tabella riempita da un protocollo di routing. L'idea di **IPA** (*Intelligent PAckets*) è diversa: il pacchetto **porta con sé** una piccola rete neurale, e ogni nodo che attraversa la esegue per decidere l'uscita.

Niente segnalazione, niente tabella condivisa: la decisione nasce dai dati che il nodo ha sotto mano in quel momento.



Il modello è una rete piccola: quattro tipi di ingresso, due strati nascosti da quattro neuroni, sette uscite. In tutto **319 pesi**. Deve stare in un header ed essere eseguibile in decine di nanosecondi.

1.2 Dove gira: XDP

Il codice gira in **eBPF** agganciato a **XDP**, il punto più precoce in cui il kernel Linux permette di toccare un pacchetto: prima che venga costruita la struttura `sk_buff`, quindi prima di quasi tutto il resto dello stack di rete.

Questo impone un vincolo forte, che spiega metà delle scelte di progetto: un programma eBPF deve essere **dimostrato sicuro** da un componente del kernel chiamato *verificatore* prima di poter girare. Niente cicli di lunghezza ignota, niente accessi non controllati alla memoria, niente ricorsione. Tutto ciò che nel documento sembra contorto — cicli srotolati, tetti dichiarati a priori, tabelle al posto di calcoli — nasce da lì.

1.3 Le tre pipeline

Lo stesso modello è implementato in tre modi, per misurare un compromesso.

	i pesi stanno	**cambiare modello**
P1 hardcoded	scritti nel codice C	ricompilare
P2 template	in una mappa	scrivere la mappa
P3 modular	in una mappa, uno strato per programma	scrivere la mappa

P1 è veloce perché il compilatore vede i pesi e li semplifica; è rigido perché ogni modello nuovo richiede una ricompilazione. P2 e P3 sono l'opposto. Questo quaderno misura il compromesso.

P1 si costruisce **prima** (*ahead of time*, AOT): il C con i pesi scritti dentro si compila con clang su una macchina qualunque, e al nodo arriva l'oggetto pronto, che un piccolo caricatore in C apre e attacca. Il caricatore lascia vuote le tabelle e le «appunta» nel file system del kernel; il piano di controllo in Python le riempie con le interfacce vere del nodo, poi dice al caricatore di attaccare il programma. Ogni cifra di P1 in questo quaderno è di quell'oggetto, attraverso un modulo solo (`ipa/p1_aot.py`), e anche il controllo rapido `execute_pipeline.py -method hardcoded -verify-only` carica quell'oggetto: lo genera, lo fa passare dal verificatore e lo esegue una volta, senza agganciarlo.

CAPITOLO 2

La matematica che sta dentro il kernel

2.1 Perché i numeri sono interi

Nel kernel non ci sono numeri in virgola mobile: eBPF lavora con interi. Il modello però è addestrato in virgola mobile. Serve una conversione, e si chiama **quantizzazione**.

Ogni peso reale w diventa un intero a 8 bit:

$$w_{\text{int}} = \text{round}(w \cdot s)$$

dove s è il **fattore di scala**, uno solo per tutta la rete.

Perché il risultato stia in un byte con segno serve

$$s \cdot \max_i |w_i| \;\leq\; 127.$$

Sul modello depositato $\max|w| = 5{,}098$, quindi $s \leq 24{,}9$. Due scelte convivono nel progetto: $s = 24$ (il massimo intero ammesso) e $s = 16$ (la potenza di due più vicina per difetto, che rende le divisioni degli shift).

> Superare quel limite non dà un errore: dà un *overflow silenzioso*. I pesi grandi si troncano e il modello sbaglia senza che nulla lo segnali. Per questo i test distinguono le scale *rappresentabili* da quelle che saturano, e misurano solo sulle prime.

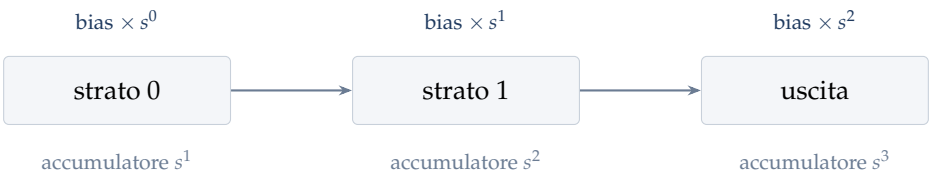
2.2 I bias vanno riportati nelle unità dell'accumulatore

Quando moltiplichiamo un ingresso per un peso quantizzato, il risultato porta un fattore s in più rispetto al valore reale:

$$x \cdot w_{\text{int}} \;=\; x \cdot w \cdot s.$$

Se impiliamo gli strati, quel fattore si accumula. Dopo $\ell + 1$ moltiplicazioni l'accumulatore dello strato ℓ porta $s^{\ell+1}$.

Ma il **bias** è memorizzato come un peso qualsiasi, cioè con un solo fattore s. Sommarlo così com'è significherebbe sommare una quantità sistematicamente troppo piccola. Va riportato nelle unità dell'accumulatore:



La regola è semplice: allo strato ℓ (contando da zero) il bias va moltiplicato per s^ℓ. In P1 il moltiplicatore si calcola alla generazione del codice e il bias entra già scalato come letterale; in P2 e P3 il programma lo calcola a runtime dalla scala del modello.

Perché i test spengono i link

Con **tutti i link attivi** gli accumulatori sono grandi, e anche un bias sottopesato di $s^2 = 576$ volte non sposterebbe mai il massimo. Basta spegnere qualche link perché gli accumulatori si riducano e un errore del genere decida l'esito. Per questo le prove di correttezza variano lo stato dei link, non solo il TTL.

2.3 Il TTL: dividere il prodotto, mai il valore

Il modello è addestrato con il TTL *normalizzato*, cioè ttl/T con $T = 30$ il valore iniziale. Con i numeri interi

$$\frac{\text{ttl}}{30} = 0 \qquad \text{per ogni ttl} < 30,$$

e la feature sparirebbe. La divisione va quindi applicata al *prodotto*:

$$\left\lfloor \frac{\text{ttl} \cdot w_{\text{int}}}{T} \right\rfloor \quad \text{e non} \quad \left\lfloor \frac{\text{ttl}}{T} \right\rfloor \cdot w_{\text{int}}.$$

Nel C generato si legge esattamente così:

```
(((__s64)_ttl * 11LL) / 30LL)
```

La scala T è del **modello**, non del codice: un modello addestrato su ttl/16 va eseguito con 16. Viaggia quindi nel descrittore del modello (il campo `scale` di `struct feat_ent`, un byte); P1 la compila come letterale, P2 e P3 la leggono dalla mappa a ogni pacchetto, e con 0 ricadono sul default compilato.

In P2 e P3 il divisore è quindi una variabile, e **BPF non ha la divisione con segno**: il numeratore invece il segno ce l'ha, perché i pesi possono essere negativi. Il datapath divide il valore assoluto senza segno e rimette il segno, con troncamento verso lo zero — lo stesso arrotondamento del riferimento Python, perché sui negativi le due implementazioni coincidono solo se arrotondano allo stesso modo.

CAPITOLO 3

Le classi non sono porte

3.1 Che cosa significa una classe

La rete produce sette numeri; il più grande vince. Ma *che cosa significa* la classe vincente?

Il significato si può *dedurre* — «la classe 6 è lo scarto», «DROP è l'ultima classe», «la classe k esce dall'interfaccia k» — e sul modello depositato ognuna di queste deduzioni sembrerebbe funzionare, perché lì le porte sono numerate come le classi. Su un altro modello darebbe risposte plausibili e sbagliate, senza alcun errore.

3.2 Una catena esplicita

Il significato è quindi **dichiarato**, mai dedotto, e passa per una catena di quattro passi, ognuno con un responsabile diverso:



- **classe**: quello che il modello ha calcolato;
- **azione**: `FORWARD`, `DROP` o `UNUSED` — una classe mai addestrata esiste ed è dichiarata tale;
- **porta logica**: un numero che il *modello* conosce;
- **ifindex**: l'interfaccia vera, che solo il *nodo* conosce e che è diversa su ogni macchina.

Nel modello depositato la dichiarazione è:

classe	azione	porta logica
0–4	FORWARD	0–4
5	DROP	—
6	UNUSED	—

Classe e porta sono due spazi di indici distinti anche nel datapath: il contatore per classe si scrive con l'indice della classe, la tabella delle uscite si legge con quello della porta. Sul modello depositato coincidono, ma il codice non ci conta.

> Con la semantica dichiarata, un modello diverso da quello depositato **smette di funzionare rumorosamente** invece di continuare a girare dando risposte plausibili e sbagliate.

3.3 Ogni modello ha la sua semantica

In P2 e P3 un solo programma per architettura serve più modelli, caricati come dati. Modelli diversi possono dare alla stessa classe significati diversi, e quindi la chiave della tabella classe → azione è la coppia $(\text{model_id}, \text{classe})$.



- **Una tabella ad accesso diretto, non una tabella hash**: 256 modelli $\times$ 2 banchi $\times$ 32 classi. Una sola lettura per pacchetto, e nessun ciclo sui modelli registrati.
- **Due banchi per modello.** Ricaricare un modello scrive il banco inattivo, poi un'unica scrittura del registro cambia insieme il numero di classi e il banco in uso. Un pacchetto vede la semantica vecchia o quella nuova, mai una tabella scritta a metà.
- **Perché non dentro il registro**, che il programma ha già letto: usare la classe come indice dentro quel valore costringe il verificatore a seguirne il valore esatto attraverso tutto l'argmax, e smette di potare i cammini. Misurato: `layer_hidden` di P3 non si carica.
- **Senza un valore vivo in più.** Il primo strato di P3 sta al limite dei 512 byte di stack nelle sue versioni strumentate: la chiave viaggia allora nella variabile che dice «questo è l'ultimo strato».

La prova. Modelli costruiti in modo che il TTL scelga la classe, così ogni classe di ogni modello è raggiungibile: un modello solo, due con la stessa semantica, due con semantiche opposte su ogni classe a pacchetti alternati, un modello inesistente o rimosso, numeri di classi diversi, un modello ricaricato con un'altra semantica. **53/53** su P2 e P3. Con una tabella condivisa simulata gli stessi test falliscono dove devono: il test distingue le due cose.

Il datapath non se ne accorge: la chiave per modello è una lettura come quella per sola classe, e la latenza non cresce da 1 a 8 modelli registrati. Il prezzo è in memoria: 128 KiB fissi per pipeline, riservati per 256 modelli anche quando ne è caricato uno solo.

CAPITOLO 4

Il motore non è lo scenario

4.1 Le dimensioni vengono dalla configurazione

Il modello depositato è addestrato su **Germany50**, una rete di 50 router usata come riferimento in letteratura, con due host attaccati. Due numeri ne descrivono la forma:

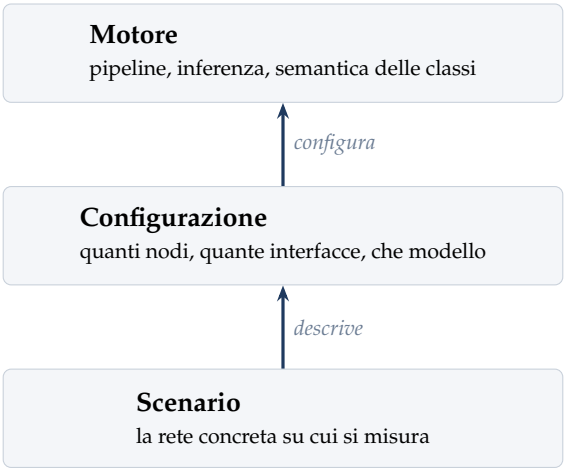
$$n_{\text{nodi}} = 52, \qquad n_{\text{interfacce}} = 6.$$

Il primo è $50 + 2$. Il secondo è il **grado massimo** della rete: il nodo con più vicini. Non è il grado di un nodo qualsiasi — tutti gli altri riempiono di zeri gli slot che non usano.

Questi numeri determinano la larghezza dell'ingresso:

$$n_{\text{in}} = \underbrace{6}_{\text{link}} + \underbrace{6}_{\text{iface}} + \underbrace{1}_{\text{ttl}} + \underbrace{52}_{\text{nodo}} = 65.$$

Il motore non li conosce. Il progetto è diviso in tre strati:



La dipendenza punta **solo verso l'alto**. Il motore non sa che cosa sia Germany50.

Le dimensioni si risolvono in ordine: una variabile d'ambiente, poi un file di sistema, poi il blocco `trained_on` del descrittore del modello. Se nessuna delle tre risponde, il motore **si ferma** e dice dove ha cercato.

> Un default comodo è il modo più efficace di nascondere un'assunzione. Preferire un errore esplicito a un valore plausibile è una scelta di progetto: il valore plausibile non si manifesta mai come errore, solo come risultato sbagliato.

4.2 La prova

Il motore genera un datapath corretto per cinque topologie che non hanno niente a che vedere con Germany50:

iface	nodi	strati	n_{in}	n_{out}	vettore link
3	8	4-4	15	4	`v[3]`
4	16	6	25	5	`v[4]`
5	24	4-4-4	35	6	`v[5]`
2	6	3-3	11	3	`v[2]`
6	52	4-4	65	7	`v[6]`

Nessuna riga di C scritta a mano cambia fra una riga e l'altra. `test_forwarding.py -sweep`: **34/34**.

CAPITOLO 5

Il modello non è quello depositato

5.1 La domanda

Un datapath che calcola la cosa giusta su un modello può calcolarla per caso: perché quel modello ha una forma particolare, perché i suoi pesi cadono in un intervallo comodo, o perché una costante scritta nel codice coincide con un suo parametro. Con un modello solo, la differenza fra "corretto" e "corretto qui" non si vede. Servono altri modelli.

5.2 I modelli sintetici

Otto scenari generati, con pesi casuali e nessun rapporto con il checkpoint depositato. Coprono un asse ciascuno:

Scenario	Forma	Pesi	scala del TTL
`ones`	11-2-3	33	8
`small`	15-4-4	84	16
`mixed`	18-6-5	149	30
`ipa_like`	65-4-4-7	319	30
`ipa_ttl16`	65-4-4-7	319	16
`sparse`	65-4-4-7	319	30
`deep`	65-4-4-4-7	339	30
`large`	117-8-8-9	1 097	64

Il generatore non produce solo numeri a caso: rispetta la **struttura** delle feature, che è la parte che un datapath può sbagliare. Ogni feature dichiara il suo tipo — `real` (un valore in un intervallo, che il datapath porta come intero diviso per una scala), `integer`, `binary` (un vettore di 0/1), `onehot` (esattamente un 1) — con i suoi estremi, la sua scala e i suoi vincoli. Ogni scenario porta con sé **1 000 vettori d'ingresso**, nella versione intera e in quella a virgola mobile, e le predizioni attese per entrambe.

Gli artefatti non stanno in git: sono rigenerabili byte per byte a parità di seme, e il test li rigenera da solo se non li trova.

5.3 Il confronto a quattro vie

Compilare e passare il verificatore dimostra che il programma esiste, non *quale classe sceglie*. E due confronti non bastano, perché un disaccordo ha più di una causa possibile e le cause chiedono rimedi diversi. Le vie sono quattro:

Confronto	Che cosa isola
float vs int8 (Python)	l'errore di **quantizzazione**: il modello a interi decide diversamente da quello a virgola mobile. Può legittimamente non essere 100%.
int8 (Python) vs int8 (riferimento)	**controprova**: due implementazioni intere indipendenti del progetto devono concordare fra loro, o non è chiaro quale sia il riferimento.
int8 (riferimento) vs int8 (eBPF)	se il datapath è esatto **rispetto alla configurazione che gli è stata data**.
int8 (Python) vs int8 (eBPF)	l'errore di **implementazione**. Deve valere 100%: un solo disaccordo qui è un difetto del datapath.

C'è anche un quinto confronto che non ha bisogno del kernel: `p1_c_eval.py` legge il C generato per P1 *come testo* e lo valuta con le regole degli interi del C, logit per logit, contro il riferimento Python. I numeri vengono dai letterali del sorgente, mai dalla lista dei pesi: un generatore che emettesse un moltiplicatore sbagliato darebbe un'altra risposta. Fatto girare su codice alterato apposta, se ne accorge.

Perché non i 1 000 vettori depositati

Il datapath non accetta un vettore d'ingresso arbitrario: lo *costruisce* da ciò che ha sotto mano — le mappe dense, il TTL del pacchetto, la porta d'ingresso risolta, l'indice del nodo. Un one-hot `node` diverso a ogni campione vorrebbe dire riconfigurare il nodo a ogni campione.

Si campiona quindi nello spazio che il datapath sa davvero esprimere, e il vettore Python si costruisce allo stesso modo. Confrontare due implementazioni su ingressi che una delle due non potrebbe mai ricevere non direbbe niente su nessuna delle due. Per la stessa ragione il banco comunica al kernel la porta d'ingresso che il riferimento usa: `BPF_PROG_TEST_RUN` senza contesto fa vedere al programma sempre l'interfaccia di prova del kernel.

5.4 Il risultato

Trecento ingressi per scenario, seme fisso.

Scenario	float vs int8	int8 Python vs eBPF (P1)	decisioni
`ones`	100,00 %	**100,00 %**	300/300
`small`	98,00 %	**100,00 %**	300/300
`mixed`	100,00 %	**100,00 %**	300/300
`ipa_like`	97,67 %	**100,00 %**	300/300
`ipa_ttl16`	98,00 %	**100,00 %**	300/300
`sparse`	99,33 %	**100,00 %**	300/300
`deep`	98,67 %	**100,00 %**	300/300
`large`	99,67 %	**100,00 %**	300/300

Equivalenza numerica esatta. P1 la dà su 8 scenari su 8, P2 e P3 su 7 su 7 (`large` non si applica: supera la tabella dei pesi di P2 e le uscite di P3). La colonna di sinistra — fra 97,67 % e 100 % — è la quantizzazione, cioè una proprietà del modello e della scala scelta, non un difetto.

La scala del TTL viene letta davvero. Sugli scenari con scala diversa da 30 la scala dichiarata sposta le decisioni (41 casi su 300 per `ipa_ttl16`, 89 per `small`). Controllo negativo in P2 e P3: riscritta nella mappa del nodo la scala al default, il datapath si stacca dal riferimento *esattamente* su quei casi, e con la scala giusta torna identico.

Un'affermazione di genericità — "il descrittore si legge a runtime, quindi qualunque modello va bene" — va verificata **voce per voce**: la forma, il numero di feature, gli offset e la scala sono quattro affermazioni, e ognuna ha bisogno del suo modello di prova.

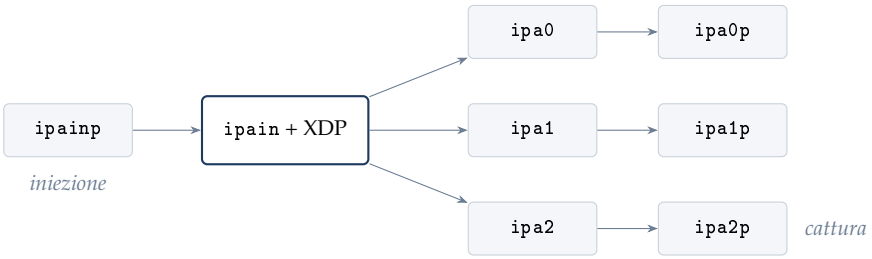
CAPITOLO 6

Dalla chiamata di sistema al kernel vero

6.1 Due banchi

Il primo banco è una chiamata di sistema, `BPF_PROG_TEST_RUN`: si dà al programma un buffer, lo si esegue *N* volte, e il kernel riporta quanti nanosecondi in media. È utile e onesto per confrontare le pipeline fra loro, ma non tocca un'interfaccia, non ha un driver, non consegna niente. Il programma può dire *"redirigerei sulla porta 3"* senza che nessun pacchetto ne esca.

Il secondo banco è fatto di coppie di interfacce virtuali (`veth`) create sull'host: abbastanza vicino al vero da far girare XDP nella modalità in cui gira su un nodo.



Si inietta un frame da un capo, il programma XDP gira davvero, e si legge da quale interfaccia è uscito.

Due dettagli che sembrano minori e non lo sono

Modalità di attacco. XDP si può agganciare in due modi: *native*, dentro il driver, prima che esista la `sk_buff`; oppure *generic*, dopo. Il default è native, la modalità *davvero in effetto* viene riletta dal kernel dopo l'aggancio, e un fallimento è un errore, non una retrocessione silenziosa: una misura fatta in generic senza saperlo non si potrebbe attribuire a niente.

Il vincolo delle veth. Redirigere *verso* una veth funziona solo se il lato ricevente ha a sua volta un programma XDP agganciato: il kernel alloca le code necessarie solo in quel caso. Senza, il redirect fallisce in silenzio e il pacchetto sparisce — indistinguibile da un modello che decide di scartare tutto. Il banco aggancia quindi un programma minimo a ogni capo.

6.2 Come il test evita di essere vuoto

Un test che non può fallire non è un test. Tre scelte del banco servono a questo:

- **Cerca un ingresso per ogni classe**, invece di spazzare alla cieca. Con tutti i link attivi il modello depositato risponde sempre con la stessa classe per qualunque TTL: variare il solo TTL ripeterebbe un caso solo. È `link_state` la feature che sposta la decisione.
- **Filtra la cattura** sui campi che il datapath non riscrive, e disabilita IPv6 sulle interfacce del banco: una veth appena creata manda traffico suo (IPv6 multicast), che altrimenti potrebbe essere contato come la risposta della pipeline.

- **Ha un ingresso dedicato**, separato da tutte le porte: se l'ingresso fosse anche una porta d'uscita, un frame inoltrato su quella porta non si distinguerebbe da quello appena iniettato.

E lo scarto si verifica dal *contatore*, non dal silenzio: "non è arrivato niente" è anche ciò che si osserva quando il redirect si perde.

6.3 Il risultato

Prova	**esito**
consegna su rete di veth, tutte le pipeline, 6 classi su 7	29/29
consegna della P1 come si deploya (oggetto AOT, mappe appuntate)	11/11
cinque topologie diverse	34/34

La settima classe è `UNUSED`: dichiarata, mai addestrata, e non ci sono ingressi che la raggiungano.

CAPITOLO 7

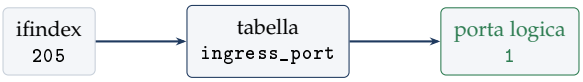
Gli ingressi che dipendono dal nodo

Due dei quattro ingressi del modello non vengono dal pacchetto ma dal nodo: *da quale interfaccia è arrivato* il pacchetto, e *quale nodo* lo sta elaborando. Tutti e due si risolvono con una tabella scritta dal piano di controllo quando il nodo entra in servizio.

7.1 L'interfaccia d'ingresso: una porta logica

L'ingresso è un one-hot di sei posizioni. Il kernel però identifica le interfacce con un `ifindex` assegnato in ordine di creazione: su un sistema appena avviato valgono 2, 3, 4; su una macchina che ha creato qualche interfaccia virtuale 205, 217, 229. Usarlo come posizione nel vettore non accenderebbe mai un bit.

Il datapath lo traduce allora in **porta logica** con una tabella a runtime, riempita dal piano di controllo leggendo le interfacce del nodo:



È il **simmetrico** della tabella usata in uscita: là "quale interfaccia realizza la porta che il modello ha scelto", qui "quale porta è l'interfaccia da cui il pacchetto è arrivato". Entrambe sono fatti del nodo, entrambe si risolvono quando il nodo esiste. Un'interfaccia che non è una porta del modello non ha voce, e il one-hot resta spento.

7.2 L'identità del nodo

L'ingresso più grande del modello è *quale nodo sta elaborando il pacchetto*: su Germany50 un one-hot di 52 posizioni, **208 dei 319 pesi**.

Non si può prendere dal pacchetto. Il campo `model_id` dell'intestazione dice *quale modello* il pacchetto chiede di usare, e viaggia identico dal mittente alla destinazione: usato come indice del nodo, ogni nodo della rete accenderebbe la stessa posizione.

L'indice sta quindi in una tabella `node_id`, scritta quando il nodo entra in servizio: dalla variabile d'ambiente `IPA_NODE_ID`, o dalla tabella nome $\rightarrow$ indice della topologia. L'indice deve seguire l'ordine dei nodi usato in addestramento, perché le decisioni significhino qualcosa sulla rete vera.

La tabella è di tipo *hash* e non *array*, e non è un dettaglio. Un array in questo ambiente è preallocato e riempito di zeri, quindi una lettura riesce **sempre**: "non configurato" e "sono il nodo 0" tornerebbero lo stesso valore. Con un hash, finché nessuno scrive non c'è voce: il vettore resta spento, ed è la risposta onesta.

7.3 La prova che contano

Stesso programma compilato, due esecuzioni che differiscono solo per la tabella del nodo:

esecuzione	**esito sul pacchetto**
senza indice di nodo	scartato
indice di nodo $= 7$	non scartato

Nient'altro cambia fra le due: l'ingresso sposta la decisione. Per l'interfaccia d'ingresso lo stesso lo mostra la consegna su veth, dove gli ingressi che raggiungono ciascuna classe dipendono dalla porta da cui il pacchetto entra.

Il prezzo sono due letture di tabella per pacchetto: P1.5 ne fa 6 in tutto, P3 da 23 a 54 secondo il modello. In P1 la tabella dell'interfaccia costa meno istruzioni dello `switch` a sei casi che sostituisce: una lettura costa meno di una catena di salti.

CAPITOLO 8

Il verificatore paga i cammini

8.1 Che cosa fa il verificatore

Prima di accettare un programma eBPF, il kernel lo fa controllare dal **verificatore**. Il verificatore non esegue il programma su un pacchetto vero: lo percorre *su tutte le strade possibili*, per essere sicuro che nessuna faccia qualcosa di proibito. Ogni `if` è un bivio, e il verificatore prende tutti e due i rami. Ha però un budget: **un milione di passi**. Finito il budget rifiuta il programma, per quanto corto sia:

```
BPF program is too large. Processed 1000001 insn
```

Dimensione del programma e costo di verifica sono quindi **monete diverse**. Due scelte del codice servono a spendere poco della seconda.

8.2 Il primo strato: feature fuori, neuroni dentro

Il primo strato si calcola sommando, per ogni neurone, i contributi di ogni feature dichiarata nel descrittore. L'ordine ovvio — neuroni fuori, feature dentro — chiederebbe *di che tipo* e *di che larghezza* è ogni feature una volta per neurone, cioè $8 \times 4 = 32$ volte per pacchetto, e ogni domanda è un bivio. Ma tipo e larghezza non dipendono dal neurone. Il codice di P2 e P3 gira quindi i cicli al contrario:

```
per ogni feature f (4 volte)
    di che tipo e' f?            <- 4 volte, non 32
    di che larghezza e' f?       <- 8 volte, non 64
    per ogni neurone j (8 volte)
        accumula                 <- nessun ramo qui dentro
```

Il risultato non cambia di un bit: cambia solo l'**ordine** in cui i termini si sommano, e la somma fra interi a 64 bit in complemento a due è associativa e commutativa, anche quando trabocca. È questo margine che permette a P3 di compilare il tetto sulle code a 8 e alle versioni strumentate di P2 e P3 (quelle che contano le letture di tabella) di caricarsi.

8.3 La ReLU: un bivio per neurone

Il verificatore paga ogni `if` due volte. Una ReLU scritta con un `if` ne mette uno in ogni neurone nascosto, e con quella scrittura due cose non si caricherebbero:

- **P2 con tre o più strati nascosti**;
- **P1 oltre qualche centinaio di pesi**, e prima la forma larga.

Non per la lunghezza: il programma P2 a tre strati ha circa 16 000 istruzioni, ma il verificatore ne percorrerebbe un milione, rileggendo gli stessi pezzi di codice molte volte, una per ogni strada.

Dove stanno i bivi

Ogni neurone nascosto applica la ReLU: *se il valore è positivo tienilo, altrimenti metti zero*. Scritta nel modo più diretto:

```
h = a > 0 ? a : 0;        // un if: due strade
```

Un bivio per neurone. Con tre neuroni in fila le strade sono

$$2 \times 2 \times 2 = 8,$$

una per ogni combinazione di "positivo / negativo":

neurone 1	neurone 2	neurone 3	
$+$	$+$	$+$	strada 1
$+$	$+$	$-$	strada 2
$+$	$-$	$+$	strada 3
	…		…
$-$	$-$	$-$	strada 8

Con i 16 neuroni del modello largo di P1 sono $2^{16} = 65\,536$ strade. Il verificatore ha un meccanismo per riconoscere due strade "equivalenti" e percorrerne una sola (la *potatura*), ma qui riesce a riunirne poche: i numeri qui sotto dicono che la gran parte delle strade viene percorsa per intero.

Il verificatore scrive quanti passi ha fatto, se glielo si chiede (`check_verifier_stats.py`, statistiche con `log_level = 4`, che prova ogni forma con le due scritture della ReLU). Con il bivio il P1 largo, 3 486 istruzioni, arriva al milione: **287 volte** la sua lunghezza. P2 a *uno* strato è già a 799 563 passi, l'80% del budget: un margine così piccolo che una differenza qualsiasi fra due kernel basta a superarlo.

La ReLU senza bivio

La ReLU si può calcolare con sola aritmetica, senza `if`:

```
m = a >> 63;              // tutti 1 se a e' negativo, tutti 0 altrimenti
h = a & ~m;               // a se positivo, 0 se negativo
```

Lo spostamento a destra di 63 posizioni di un intero con segno a 64 bit "ricopia" il bit del segno su tutti i bit. Due esempi, mostrati su 8 bit per leggibilità (a 64 bit è identico):

	$a = 5$	$a = -3$
a in bit	00000101	11111101
$m = a \gg 63$	00000000	11111111
$\sim m$	11111111	00000000
$a \& \sim m$	$00000101 = 5$	$00000000 = 0$

Con a positivo la maschera lascia passare tutto e resta a; con a negativo la maschera azzera tutto. È la ReLU, **bit per bit la stessa**, ma senza bivio: il verificatore ha una strada sola da percorrere.

Il dettaglio che la fa funzionare

Scritta così e basta, non cambia niente: clang riconosce che `a & ~(a >> 63)` è "il massimo fra a e 0" e, per ottimizzare, **rimette il bivio**. Nel bytecode compilato i salti tornano tutti (136 contro 120 nel P1 largo). Serve una riga che impedisca a clang di vedere dentro la maschera:

```
static __always_inline long long ipa_relu(long long a) {
    long long m = a >> 63;
    asm volatile("" : "+r"(m));   // barriera: nessuna istruzione, m invariato
    return a & ~m;
}
```

L'`asm` è vuoto: non produce nessuna istruzione e non cambia m. Dice solo a clang "di m non sai niente", e clang non può più riconoscere il massimo. La funzione è la stessa in P1 (`gen_full_c.py`), in P2 (il leaf di `ebpf_template_arch.py`) e in P3 (`ebpf_modular.py`). P3 si caricherebbe anche con il bivio, perché ripete un solo strato generico: usa la stessa funzione perché le tre pipeline calcolino l'attivazione con le stesse istruzioni, e il confronto fra loro resti alla pari.

8.4 La ReLU senza bivio: il risultato

Passi del verificatore, su un budget di 1 000 000:

	ReLU con bivio	**ReLU senza bivio**
P2, 1 strato	799 563	120 452
P2, 2 strati	417 292	120 557
P2, 3 strati	*rifiutata*	127 906
P2, 6 strati	*rifiutata*	136 642
P2, 20 strati	*rifiutata*	201 528
P1 larga, 1×16 (∼1 200 pesi)	*rifiutata*	5 698
P1 profonda, 8×9 (∼1 300 pesi)	229 039	8 516
P3, `layer_first`	27 183	26 755
P3, `layer_hidden`	186 489	27 979

Senza bivio P2 carica fino a venti strati, e a sei ne usa il 14% del budget. P1 carica il tier da ∼1 200 pesi in tutte le forme. Lo strato che P3 ripete (`layer_hidden`) costa al verificatore 17 volte la sua lunghezza invece di 117. `layer_first` quasi non cambia, perché la sua ReLU è una parte piccola di un programma dominato dalla lettura degli ingressi.

Il limite di P2 non è il verificatore

Oltre venti strati è **clang** a fermarsi, prima ancora del kernel:

```
LLVM ERROR: Branch target out of insn range
```

Un salto eBPF può arrivare al massimo a 32 767 istruzioni di distanza. I controlli all'inizio del programma (*se il pacchetto non va bene, esci*) saltano fino in fondo, e ogni strato allunga il programma di circa mille istruzioni: a venti strati il programma ne ha 32 388, appena sotto. Il terzo limite è la tabella dei pesi, 1 024 posti: 37 strati al massimo a larghezza 4, 7 a larghezza 8.

8.5 La ReLU senza bivio: che cosa costa

Il calcolo della rete è identico: i test che confrontano il C di P1 con il riferimento Python danno gli stessi logit su ogni modello sintetico (`test_synthetic_models.py`), e `p1_c_eval.py` valuta anche `ipa_relu` leggendone il corpo. Cambiano invece le istruzioni: circa tre per neurone, sempre eseguite, al posto di un salto. Nel banco a pacchetto ripetuto un salto sarebbe quasi gratis, perché lo stesso pacchetto passa milioni di volte e il processore indovina

sempre da che parte va: lì le tre istruzioni in più si vedono, soprattutto in P1, il programma più corto. In P2 e P3 la ReLU è una parte minima di un programma molto più grande. In cambio P1 carica modelli fino a quattro volte più grandi, P2 fino a venti strati, e le tre pipeline calcolano l'attivazione con le stesse istruzioni.

Il verificatore non paga le istruzioni ma le **strade**: togliere un bivio vale più che togliere cento istruzioni. E un programma che lavora all'80% del budget di verifica si carica su un kernel e non su un altro: il margine va misurato, non presunto.

CAPITOLO 9

Il test reale: quanto costa un pacchetto al nodo

9.1 La domanda, in parole semplici

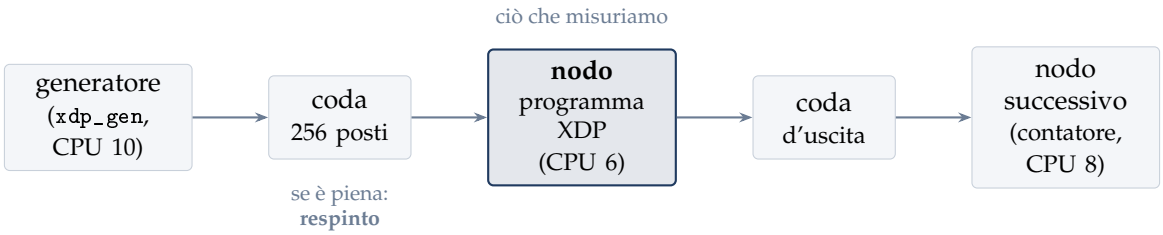
Un nodo della rete riceve pacchetti, per ognuno esegue la rete neurale, decide da quale porta farlo uscire, e lo spedisce. La domanda è: **quanti pacchetti al secondo regge il nodo**, e **quanto tempo spende su ognuno**?

Un'immagine per tutto il capitolo: la cassa del supermercato

Il nodo è un **cassiere**. I pacchetti sono i **clienti**. Il generatore di traffico è la porta da cui entrano i clienti, e il collegamento virtuale fra generatore e nodo è il **nastro** su cui i clienti aspettano: ha 256 posti, e quando è pieno chi arriva viene mandato via.
Per sapere quanto spende il cassiere per cliente si fanno arrivare più clienti di quanti ne riesca a servire, così che il nastro non sia mai vuoto, e si conta quanti ne serve in un secondo. Se ne serve 5 milioni al secondo, spende 1/5 000 000 di secondo a cliente, cioè 200 nanosecondi.

9.2 Com'è fatto il banco

Tutto sta sulla macchina di laboratorio, e ogni parte ha il suo core fisico, a frequenza fissa (capitolo 10):



- **Il generatore** (`xdp_gen`, sezione 9.4) crea pacchetti uguali a quelli veri: stesso formato, stesso header IPA, stesso modello da eseguire. Un thread, che scrive nell'unica coda del nodo. Generatore e nodo successivo stanno su P-core; il nodo su un P-core, un E-core o un LP E-core, secondo la domanda (sezione 9.11).
- **La coda d'ingresso** è quella di un *veth*, il cavo virtuale di Linux: 256 posti, e la dimensione non si può cambiare.
- **Il nodo** è una delle pipeline caricata in XDP. È l'unica cosa che si vuole misurare.
- **Il nodo successivo** conta i pacchetti arrivati e li butta, su un core suo, così che sul core del nodo resti solo il lavoro del nodo.

Oltre alle cinque pipeline il banco misura, nella stessa sessione, `rxonly`: un programma che prende il pacchetto dalla coda e lo butta senza guardarlo. Dice quanto costa *soltanto ricevere*: è il pavimento sotto il quale nessuna pipeline può scendere.

9.3 Come si misura, e perché così

Un numero affidabile richiede quattro accortezze. Nessuna riguarda le pipeline: senza, tutte farebbero sembrare il nodo diverso da com'è.

1. La finestra stazionaria

Il banco avvia il generatore, aspetta che stia trasmettendo, lascia assestare la coda, legge i contatori, aspetta 300 ms, li rilegge, e solo allora ferma il generatore. Il rate è la differenza fra le due letture divisa per il tempo fra le due letture: l'avvio e l'arresto restano fuori.

2. Letture brevi

Ogni lettura (generatore più nodo) deve durare meno di 3 ms; se dura di più viene rifatta, e le si attribuisce l'istante a metà. Un processo interrotto a metà lettura attribuirebbe ai contatori un istante sbagliato di tutta l'interruzione. Un test senza kernel lo verifica con un generatore simulato.

3. I respinti contano

Quando la coda d'ingresso è piena il collegamento rifiuta il pacchetto («respinto»). Un pacchetto respinto è un pacchetto che il nodo non ha preso, perché non svuotava la coda abbastanza in fretta: **è una perdita del nodo**. Il banco conta separatamente i **respinti** (mai entrati) e i **persi dopo l'ingresso** (entrati ma non usciti).

4. Il tempo di CPU, non il tempo che passa

Il banco misura anche quanto il core del nodo è occupato, nella stessa finestra in cui conta i pacchetti, e dà il costo come **tempo di CPU per pacchetto**: occupazione per il tempo diviso i pacchetti. Se il nodo lavora il 100% del tempo coincide con 1 diviso la capacità; se a tratti aspetta, toglie l'attesa. Con `xdp_gen` il nodo lavora al 100%, e le due misure coincidono.

Nella stessa lettura il banco legge anche i **contatori hardware** del core del nodo: istruzioni e cicli. Divisi per i pacchetti dicono quante istruzioni costa un pacchetto e quante ne esegue il core per ogni ciclo (l'IPC). Il tempo di CPU dice *quanto* costa un pacchetto; istruzioni e IPC dicono *perché*.

9.4 Il generatore

`xdp_gen`: pacchetti come da una scheda di rete

Il generatore (`ipa/test/xdp_gen.py`) consegna al cavo virtuale pacchetti già nel formato di XDP, con `BPF_PROG_TEST_RUN` in modalità *live frames*: il kernel li prepara in una sua riserva di pagine, con i 256 byte liberi davanti che XDP vuole, e li spedisce davvero. Niente skb e niente copia: il pacchetto arriva al nodo come arriverebbe da una scheda di rete con XDP nativo, e il nodo paga solo il proprio lavoro. Un solo thread offre fino a circa 19 milioni di pacchetti al secondo, ed è così che si usa: un thread per ogni coda del nodo (sezione 9.8).

Ogni misura è **una sola chiamata** al kernel, interrotta da un segnale a fine finestra. Ogni chiamata, infatti, parte e finisce con una pausa del generatore di circa 12 ms dentro il kernel: con più chiamate per finestra il generatore tacerebbe una parte del tempo, e il nodo svuoterebbe la coda e dormirebbe. Le statistiche di scheduling del thread del nodo lo controllano: a coda piena lavora il 100% del tempo.

Il ritmo e il timbro

Per il test al crescere del traffico (sezione 9.9) serve un ritmo fissato, e `xdp_gen` non ne ha uno suo: spinge al massimo. Il ritmo lo tiene il suo programma, che a ogni giro guarda l'orologio: se non è ancora il momento scarta il pacchetto (la pagina torna subito alla riserva) e riprova; quando è il momento lo spedisce, e fissa il prossimo istante un intervallo più in là. I pacchetti partono a distanza regolare, non a raffiche: una raffica da 256 riempirebbe da sola la coda e farebbe perdere pacchetti sotto capacità. Nello stesso istante il programma scrive nel pacchetto un timbro con l'ora di partenza, e il nodo successivo misura il ritardo. Il ritmo è esatto fino a circa 11 milioni di pacchetti al secondo.

L'alternativa: `pktgen`

`pktgen` è il generatore del kernel: fabbrica i pacchetti come lo stack di rete, in una *skb* con 64 byte liberi davanti, e il cavo virtuale li **ricopia** in una pagina nuova prima di consegnarli al nodo. La copia la paga il nodo, e su una scheda vera non esisterebbe.

Per questo il banco usa `xdp_gen`, e tutte le cifre di questo capitolo sono con `xdp_gen`; `pktgen` resta disponibile per i percorsi che lo richiedono.

9.5 Perché un core vero, tutto suo

Il nodo deve avere un core fisico suo, che non gli venga mai tolto, e il conto che lo spiega è semplice. La coda d'ingresso ha 256 posti. A 1,5 milioni di pacchetti al secondo un pacchetto arriva ogni $1/1{,}5 \cdot 10^6 \approx 0{,}67$ microsecondi, quindi la coda si riempie in

$$256 \times 0{,}67\ \mu\text{s} \;\approx\; 170\ \mu\text{s}.$$

Se il core del nodo si ferma per più di 170 microsecondi — perché il sistema lo dà a un altro processo, o perché si addormenta — la coda trabocca e i pacchetti vengono respinti, *per quanto veloce sia il programma*. Nell'immagine del supermercato: il cassiere è bravissimo, ma se ogni tanto esce dalla cassa per qualche secondo il nastro si riempie e i clienti vengono mandati via. Una macchina virtuale, i cui core sono thread dell'ospite che possono sparire per qualche millisecondo, non può fare questo banco.

Per questo:

- **Core veri, ognuno col suo compito.** Generatore sul core 10, nodo sul core 6 (o 12, o 20), nodo successivo sull'8. Nessun altro processo, nessuna interruzione e nessun thread del kernel sul core del nodo (capitolo 10).
- **Il core del nodo non dorme mai.** Gli stati di sonno profondi sono spenti: il risveglio dal più profondo costa 310 microsecondi, più dei 170 che la coda concede. Sotto capacità il nodo non respinge niente.
- **Frequenza fissa e misurata**: 3 500 MHz sul core del nodo (2 500 sul LP E-core), letta dai contatori del processore durante ogni misura.

- **Generatore e nodo su core separati**, quindi il generatore non è dentro la misura: il ritmo a cui il nodo satura è quello del nodo.

Il risultato si vede ripetendo: i tre giri di una misura differiscono dell'1,3 % in mediana, e la stessa misura rifatta in un'altra sessione dell'1,8 % in mediana, al più del 7 %.

9.6 I limiti del test, con esempi

Il condizionamento della macchina toglie il rumore. Non toglie i limiti di un banco che sta tutto su un portatile: vanno conosciuti, perché ognuno dice quali numeri si possono citare e quali no.

1. Il generatore può non bastare, o intralciare

Se il nodo è più veloce di quanto il generatore riesca a produrre, il banco misura il generatore. Un thread di `xdp_gen` offre circa 19 milioni di pacchetti al secondo a massima spinta, meno a ritmo fissato e meno ancora quando i pacchetti vengono inoltrati: per questo su un P-core `rxonly` non arriva mai al suo limite, e il suo tempo di CPU è indicativo. Il banco lo riconosce dall'occupazione del nodo: un nodo che perde pacchetti lavorando meno del 90% del tempo non è limitato dal programma. Aggiungere thread sembra la soluzione, ma più thread che scrivono nella *stessa* coda si intralciano, e rallentano anche il nodo che la svuota: è la sezione 9.8.

2. Contare all'uscita può sottostimare il nodo

Il nodo successivo, che conta i pacchetti arrivati, ha i suoi core e i suoi tetti. Se si misura il nodo contando *ciò che arriva dall'altra parte*, quando il nodo è più veloce dell'uscita si misura l'uscita.

Esempio: la baseline che sembrerebbe lenta

Un core d'uscita spende circa 70 ns per ricevere, contare e restituire un pacchetto: regge al più una dozzina di milioni di pacchetti al secondo. Se due core del nodo gli mandano 25 milioni di pacchetti al secondo, la baseline ne *elabora* 25 ma all'altra parte ne arrivano 12; P2 ne elabora 6 e arrivano tutti. Contando gli arrivi, la baseline sembrerebbe più lenta di P2 — impossibile, perché fa strettamente meno lavoro.
Per questo il controllo di validità del banco ("la baseline dev'essere la più veloce") conta i pacchetti *elaborati dal nodo*, la stessa misura con cui il banco calcola il costo, e non quelli arrivati all'uscita.

Con un solo scrittore per coda anche all'uscita il nodo successivo tiene il passo: con due core sul nodo serve una coda d'uscita per ciascun core, e un core d'uscita per ciascuna coda; allora arrivano tutti, anche 29 milioni al secondo (sezione 9.10).

3. Il cavo è virtuale, e tutto sta su un processore

Il "ricevere" misurato è prendere il pacchetto dalla coda di un cavo virtuale, non dalla memoria di una scheda di rete; "inoltrare" è accodarlo sul cavo successivo, non scrivere un descrittore di trasmissione. Generatore, nodo e nodo successivo sono core dello stesso chip e condividono cache e memoria. Il portatile non ha una scheda di rete cablata che supporti XDP nativo.

Esempio: i 66 ns per pacchetto della baseline sono il prezzo *di questo percorso*. Su una scheda vera sarebbero altri numeri, e le cifre assolute di questo capitolo con loro. I **confronti** fra pipeline no: il percorso è lo stesso per tutte, e si sottrae.

4. Un solo tipo di pacchetto per volta

In una misura tutti i pacchetti sono uguali: stesso modello, stesso TTL, stessa classe decisa. Il processore impara in fretta un flusso così (il predittore dei salti, la cache). Per questo la campagna misura anche un traffico con un TTL diverso a ogni pacchetto, e quindi classi diverse: costa come il pacchetto fisso, 0 [−6–4] %. Le classi si misurano anche una alla volta, ed è così che si vede che scartare costa più che inoltrare.

5. Il ritardo a basso carico contiene il generatore

`xdp_gen` spedisce a lotti di 256 frame, e il nodo viene svegliato a fine lotto: a basso carico un pacchetto può aspettare fino a una dozzina di microsecondi prima che il nodo lo veda, e la mediana del ritardo contiene quell'attesa. Lotti più piccoli la tolgono, ma il generatore regge ritmi più bassi. Il salto del ritardo a coda piena, che è ciò che il test cerca, non cambia.

6. Il punto esatto in cui comincia la perdita è instabile

Al confine fra "regge" e "perde" basta un'esitazione di pochi microsecondi per perdere un pacchetto: lo stesso punto, ripetuto, varia molto più della saturazione. Si citano quindi la saturazione e i gradini della scala, non il confine esatto.

7. Il portatile scalda

Oltre 3,5 GHz non regge una misura intera, e anche a 3,5 il processore nel suo insieme tocca la soglia di temperatura circa una volta al minuto: lo decide il firmware. Il banco registra ogni evento e conta le finestre disturbate (anche il funzionamento a batteria è un disturbo); come la campagna tratta le misure colpite lo dice il capitolo 12.

9.7 Il risultato: quanto regge il nodo

Con `xdp_gen`, un core del nodo (un P-core a 3,5 GHz), pacchetti da 64 byte, tre giri, sui 109 modelli della popolazione (capitolo 12), mediana [minimo–massimo]:

Pipeline	Mpps	ns per pacchetto	costo della rete	istruzioni	IPC
solo ricezione (`rxonly`)	19,3	40	—	383	2,5
baseline (nessuna rete)	15,0 [14,5–15,4]	66 [63–69]	—	606	2,6
P1 statica	8,4 [4,5–11,7]	119 [85–221]	53 [19–155]	1 168	2,9
P1.5	8,0 [4,5–11,6]	124 [86–222]	58 [19–156]	1 349	3,2
P2	3,4 [2,0–5,1]	292 [196–512]	226 [129–446]	4 578	4,5
P3	2,1 [1,1–3,3]	487 [307–918]	421 [240–852]	7 620	4,5

P2: 103 modelli (gli altri 6 superano il suo tetto sui pesi). `rxonly` è una misura sola, senza modello, e non satura: il suo valore è indicativo. Costo della rete = ns per pacchetto meno la baseline dello stesso run. Istruzioni e IPC per pacchetto, mediane, dai contatori hardware del core del nodo.

La **baseline** riceve, legge l'intestazione, decrementa il TTL e inoltra. Tutto ciò che le pipeline aggiungono sopra è **il costo della rete neurale**: in mediana circa 50–60 ns quando i pesi sono scritti nel programma (P1, P1.5), circa 225 ns quando la forma della rete è letta da una tabella (P2), circa 420 ns quando anche la profondità lo è (P3).

Come si legge la colonna dei nanosecondi, con un esempio. La baseline regge 15 milioni di pacchetti al secondo, cioè spende $10^9/15 \cdot 10^6 \approx 66$ ns per pacchetto; un modello che P2 regge a 3,4 milioni costa 292 ns. La differenza, circa 226 ns, è il tempo che P2 spende a eseguire la rete neurale: tutto il resto — ricevere, leggere l'intestazione, inoltrare — lo fanno tutte e due allo stesso modo. Il nodo lavora il 100% del tempo, e i cicli per pacchetto dei contatori, divisi per 3,5 GHz, ritrovano il tempo di CPU.

Le istruzioni per pacchetto dicono un'altra cosa che il programma da solo non dice: P3 *esegue* più istruzioni di P2 (7 620 contro 4 578 in mediana), pur caricandone di meno, su 103 modelli su 103. E l'IPC delle pipeline con i pesi in tabella è alto, oltre 4: è aritmetica in fila, senza bivi, che il core esegue molte istruzioni per volta.

E sotto la capacità? Sulle curve a traffico crescente (sezione 9.9), finché il traffico resta sotto la capacità di una pipeline il nodo inoltra tutto ciò che riceve. Un nodo sotto la sua capacità consegna tutto.

P1 e P1.5 si separano

Congelare il nodo vale poco ma si vede: in mediana 119 contro 124 ns per pacchetto, e la rete di P1.5 costa 1,11 [0,93–1,68] volte quella di P1, più di P1 su 89 modelli su 109.

Da che cosa il costo non dipende

- **La taglia del pacchetto non conta**: fra 64 e 1 514 byte il costo cambia di 1,2 [0,3–3,1] %. Il nodo guarda solo l'intestazione.
- **La porta d'uscita non conta**: le classi che inoltrano costano uguale, 0 [0–9] % fra la più cara e la meno cara.
- **Scartare costa di più che inoltrare**: 51 [17–63] ns in più. Quando il pacchetto viene scartato la sua pagina di memoria si restituisce sul core del nodo; quando viene inoltrato, su quello del nodo successivo. È un effetto del banco, non della pipeline.

9.8 Uno scrittore per coda

Il generatore usa **un thread per coda** del nodo, e non di più. Più thread che scrivono nella stessa coda da 256 posti devono passarsi un lucchetto e le stesse righe di cache; il core del nodo, che dalla stessa coda legge, ne subisce il rallentamento, e il nodo elabora *meno* pacchetti invece di più. È come una cassa dove tre fornitori scaricano merce nello stesso punto: il cassiere non lavora peggio, ma gli spazi sono contesi. Il danno è grande per i programmi leggeri e quasi nullo per quelli il cui limite è davvero il programma.

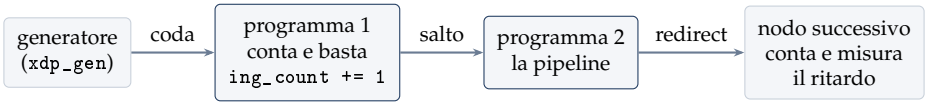
Un solo scrittore per coda, in ingresso e in uscita: più thread di generatore non aiutano, peggiorano. Con una scheda di rete vera la regola è rispettata da sé: la coda di ricezione la riempie la scheda, un solo scrittore, e i pacchetti degli altri host arrivano in fila sul cavo. Resta quello che conta, la coda piena davanti a un nodo troppo lento.

9.9 Al crescere del bit rate

La domanda: al crescere del traffico offerto la coda si riempie, il tempo di attraversamento cresce, e poi si perde. Se si perde *all'ingresso* del programma, il collo di bottiglia è il programma; se si perde *dopo*, è la trasmissione. Per saperlo bisogna contare quanti pacchetti il programma riceve e quanti ne rilancia.

Due programmi sullo stesso aggancio

Un'interfaccia porta un solo programma XDP. Due programmi sullo stesso aggancio si ottengono in **catena**, con lo stesso salto fra programmi che le pipeline usano già:



Il primo programma è attaccato all'interfaccia, conta ogni pacchetto e salta al secondo, che è il dispatcher della pipeline, identico a quello di produzione.

Perché in catena. Il kernel tiene, per ogni interfaccia, *un* programma XDP nell'aggancio nativo: è come è fatto XDP, non un limite di questo progetto. I modi di averne più di uno sono tutti catene. Quello standard è `libxdp`, che aggancia un programma *dispatcher* con alcuni posti liberi e ci inserisce gli altri; quello usato qui è il salto fra programmi (*tail call*) che le pipeline già usano, senza dipendenze in più. Il costo del salto in più si misura: il kernel tiene il tempo di ogni programma attaccato, e il banco lo legge con e senza il contatore davanti.

Così si hanno tre conteggi per ogni valore di traffico: **inviati** (dal generatore), **ricevuti** (dal programma 1) e **inoltrati** (arrivati al nodo successivo). Inviati meno ricevuti è la perdita *prima* del programma; ricevuti meno inoltrati quella *dopo*. Un terzo metodo, `rxonly`, è il contatore da solo: dice quanto regge la sola ricezione allo stesso ritmo, e separa il collo del programma da quello del banco.

Il banco misura sedici valori di traffico, da 0,5 a 12 milioni di pacchetti al secondo (da 0,26 a 6,1 Gbit/s a pacchetti da 64 byte), tre giri ciascuno. Il ritardo è quello di tutto il percorso: dal timbro che `xdp_gen` scrive nel pacchetto quando lo spedisce all'arrivo al nodo successivo.

Il risultato

Sei modelli della popolazione, dal più leggero al più pesante, sul P-core e sull'E-core: 48 curve. Ognuna sale lungo la diagonale finché il nodo regge — tutto ciò che si invia viene ricevuto e inoltrato — poi si ferma alla capacità della pipeline: il massimo inoltrato sta a **0,98 [0,88–1,03]** della capacità misurata a massima spinta sullo stesso modello sul P-core, e a **0,99 [0,94–1,03]** sull'E-core. Oltre la capacità i pacchetti si perdono **prima del programma**, alla coda d'ingresso: la quota persa prima di XDP è 1,000 in mediana. Il programma rilancia tutto ciò che riceve, e ciò che manca non è mai entrato.

Il ritardo fa quello che la domanda prevede: resta di qualche decina di microsecondi finché il nodo regge, poi salta dove la pipeline comincia a perdere. Il salto è la coda d'ingresso che si riempie: 256 posti davanti a un programma che serve un pacchetto ogni 487 ns (P3, in mediana) fanno circa 125 microsecondi di attesa, più il percorso fino al nodo successivo. Oltre, la coda non può crescere: è piena, e ciò che arriva in più viene respinto. Il ritardo si ferma e cresce la perdita. A basso carico la mediana contiene l'attesa nel generatore (sezione 9.6, punto 5).

Perché il collo di bottiglia è il programma

È quello che ci si deve aspettare, per come funziona XDP. Il programma non ha una coda sua: gira dall'inizio alla fine su ogni pacchetto, sul core che riceve, e finché non ha finito quel core non prende il pacchetto successivo. Se i pacchetti arrivano più in fretta di quanto il core li finisca, si accumulano nella coda *davanti* al programma, e quando è piena vengono respinti. La perdita all'ingresso è quindi il segno di un nodo più lento del traffico, e il nodo è il suo programma.

Resta da escludere che il limite sia la sola ricezione, che gira sullo stesso core. Lo dice `rxonly`, il contatore da solo: la sola ricezione regge circa 19 milioni di pacchetti al secondo, più di quanto le pipeline arrivino a inoltrare. Fa eccezione P1 sui modelli più leggeri, che regge oltre 9 milioni al secondo: lì comincia a perdere anche l'uscita (la quota persa prima di XDP scende fino a 0,69, il massimo inoltrato fino a 0,88 della capacità). È il limite del cavo virtuale e del core d'uscita, non della rete neurale.

Due precisazioni. Per la baseline il "programma" è quasi solo lettura dell'intestazione e redirect: il suo limite è il costo fisso di un nodo XDP, non una rete neurale. E il nodo ha **una sola coda e un solo core** per scelta, per misurare il costo per core: una scheda di rete vera distribuisce i pacchetti su più code e più core (RSS), e la capacità del nodo cresce con il numero di core, mentre il costo per pacchetto resta quello misurato qui.

9.10 Due core

Perché la capacità cresca davvero con i core, i core non devono contendersi niente di scritto a ogni pacchetto. Per questo i contatori delle pipeline (pacchetti inoltrati, persi, scartati, e uno per classe) sono **per-core**: ogni core incrementa la sua copia, senza istruzioni atomiche, e chi legge somma le copie. E ogni core del nodo ha la sua coda d'ingresso, il suo thread di `xdp_gen`, la sua coda d'uscita e il suo core d'uscita: il cavo virtuale sceglie la coda d'uscita dal numero del core che inoltra, e con una coda sola i due core del nodo scriverebbero nello stesso anello, svuotato da un solo core del nodo successivo (l'esempio della sezione 9.6, punto 2).

Sugli 8 modelli del sottoinsieme, dal più leggero al più pesante, due core elaborano il **99 [94–103] %** del doppio di uno sul P-core e il **99 [98–101] %** sull'E-core (due E-core, uno per modulo), su 40 coppie modello-pipeline per core. Un core e due core sono misure di sessioni diverse, ciascuna con il suo rumore: i valori sopra il 100 % sono quello, non un guadagno. Il secondo core non rallenta il primo, e per un nodo con N code vale N volte la capacità misurata su un core, finché generatore e scheda reggono.

> Per tutte le pipeline la perdita è **all'ingresso**: il collo di bottiglia è il programma eBPF, non la trasmissione, e la sola ricezione non perde affatto. Il ritardo cresce fino a una perdita, come previsto, e si ferma al livello della coda piena.

9.11 Il nodo sui core lenti

Il processore del portatile ha tre tipi di core: sei **P-core** veloci (Redwood Cove), otto **E-core** efficienti (Crestmont, in due moduli da quattro che dividono la cache L2 e la frequenza) e due **LP E-core**, che stanno su un'altra parte del chip, fuori dalla cache L3. Che cosa succede se il nodo gira su un core lento?

Per rispondere cambia *solo* il core del nodo. Generatore e nodo successivo restano su P-core a 3,5 GHz, così il generatore satura sempre il nodo e l'uscita non diventa il collo di

bottiglia. L'E-core gira anche lui a 3,5 GHz (il suo massimo è 3,8): a parità di frequenza, la differenza di capacità è tutta nel numero di cicli per pacchetto. Il LP E-core gira al suo massimo, 2,5 GHz. Gli altri tre core del modulo dell'E-core restano a riposo: dividono con lui cache e frequenza.

> Il codice eseguito è lo stesso su ogni core: le istruzioni per pacchetto differiscono al più dello 0,3 % fra P-core ed E-core e dello 0,7 % sul LP E-core. **Cambia l'IPC**, cioè quante istruzioni il core completa per ciclo. E non cambia allo stesso modo per tutte le pipeline.

L'E-core rallenta soprattutto la **parte fissa** del nodo: ricevere il pacchetto dalla coda, inoltrarlo, restituire la pagina. È lavoro del kernel, con salti e accessi alla memoria, e l'E-core lo fa a IPC 1,7 invece di 2,6: la baseline costa 106 ns invece di 66, con 1,56 [1,50–1,61] volte i cicli. L'aritmetica della rete neurale, invece, è una fila di moltiplicazioni senza bivi, e l'E-core la esegue quasi altrettanto bene: i cicli della sola rete costano da 0,95 (P1) a 1,13 (P3) volte quelli del P-core, in mediana. Per questo il rapporto di capacità E/P sale con il peso della pipeline: 0,63 per la baseline, 0,76 per P1, 0,82–0,83 per P2 e P3.

Il LP E-core paga anche la memoria: sta fuori dalla L3, e i pacchetti scritti dal generatore gli arrivano da lontano, con 13–21 mancate dell'ultimo livello di cache per pacchetto. Fra la frequenza più bassa e l'IPC più basso regge 0,43 [0,32–0,53] volte il P-core sulle pipeline. Le cifre modello per modello sono nel capitolo 12.

9.12 Che cosa questo numero non dice

I limiti del banco sono nella sezione 9.6. Uno in più riguarda i core:

- **Due tipi di core lenti, a una frequenza.** L'E-core è misurato a 3,5 GHz, non al suo massimo di 3,8; il LP E-core solo al suo massimo.

CAPITOLO 10

La macchina di laboratorio, sotto controllo

Tutte le misure di questo quaderno sono fatte sulla macchina di laboratorio, senza strati di mezzo: un portatile Lenovo IdeaPad Slim 5 con un **Intel Core Ultra 7 155H**, Ubuntu 24.04 installato direttamente sul disco (dual boot), kernel 6.8. Questo capitolo spiega che cosa serve perché le misure si ripetano.

10.1 Un processore che non sta fermo

Il 155H è **ibrido**: tre tipi di core diversi sullo stesso chip.

tipo	CPU	frequenza	note
P-core (prestazioni)	0–11	base 1,4, turbo 4,5–4,8 GHz	6 core fisici, due thread ciascuno
E-core (efficienza)	12–19	base 0,9, turbo 3,8 GHz	
LP E-core	20–21	base 0,7, turbo 2,5 GHz	fuori dalla cache condivisa

Lasciato a sé stesso, un banco su questa macchina finisce dove capita: una regola generica per scegliere i processori (un terzo al nodo, il resto al generatore) darebbe quattordici thread di generatore e metterebbe il nodo sui core di efficienza, e il desktop girerebbe sul gemello del core misurato. La frequenza va da 0,4 a 4,8 GHz secondo il carico, e i core si addormentano in uno stato da cui il risveglio costa 310 microsecondi: più di quanto la coda d'ingresso da 256 posti copra a un milione di pacchetti al secondo.

E il portatile **scalda**. Un solo P-core salito a 4,5–4,8 GHz porta il processore da 51 a 99 °C in due secondi, e il chip riduce la frequenza per difendersi (*throttling*).

10.2 Le condizioni, messe e tolte

`ipa/test/host_conditions.py` mette la macchina in condizioni note per la durata di un banco, e le riporta com'erano alla fine.

- **Ruoli**: nodo, nodo successivo e generatore su core fisici *diversi*, un thread per core; il gemello di ciascuno resta a riposo; il core della CPU 0 resta al sistema. Qui: nodo sulla CPU 6, nodo successivo sull'8, generatore sulla 10. Quando il nodo va su un E-core (la 12) restano a riposo anche gli altri tre core del suo modulo (13–15), che dividono con lui la cache L2 e la frequenza.
- **Frequenza fissa** sui core del banco (governor *performance*, minimo uguale al massimo), e **misurata** dai contatori del processore (APERF/MPERF).
- **Tetto sui core del sistema** (2,5 GHz): il turbo del desktop scalderebbe il chip fino a rallentare anche i core del banco.
- **Niente sonno profondo** sui core del banco: spenti gli stati con risveglio oltre un microsecondo.

- **Isolamento**: desktop, servizi, interruzioni e thread del kernel spostabili confinati sulle CPU del sistema.
- **Frequenza della cache condivisa fissa**, profilo energetico *performance*, watchdog spento.

Ogni valore si salva *prima* di cambiarlo, in un file in memoria. A fine run — anche con Ctrl-C o un errore — si riscrive tutto all'indietro; un run ucciso lascia il file, e il run successivo ripristina per primo. Un test sul kernel vero (`test_host_conditions_kernel.py`, 25 controlli) applica le condizioni, le rilegge dal kernel, verifica che un processo del desktop giri solo sulle CPU del sistema, misura la frequenza sotto carico, ripristina e confronta la macchina valore per valore con com'era — anche dopo aver ucciso il processo a condizioni applicate.

Durante ogni misura un **monitor** legge gli interventi di throttling, la temperatura, la frequenza vera del nodo e le interruzioni del firmware. Una finestra misurata fuori dalle condizioni dichiarate viene marcata e contata, non scartata in silenzio.

10.3 La frequenza scritta non è la frequenza vera

La frequenza *base* del processore, 1,4 GHz, è l'unica garantita. Il sistema la accetta e la rilegge: 1 400 minimo, 1 400 massimo. I contatori del processore dicono un'altra cosa:

> Con 1 400 MHz scritti nel sistema, il core gira a **circa 2 000 MHz** — fisso e stabile, ma non a quella frequenza. Da 3 000 MHz in su scritto e vero coincidono entro 10 MHz.

Se il monitor si fidasse del numero scritto, a 1,4 GHz marcherebbe come disturbate quasi tutte le finestre. La frequenza di riferimento è quindi quella **misurata** all'avvio, su ogni core del banco; il numero scritto resta accanto, per confronto.

10.4 Quale frequenza

Più alta la frequenza, più alto il throughput — finché il portatile regge. A 4,5 GHz il throughput smette di crescere, perché il chip si difende; a 4 GHz un run di qualche minuto porta il processore vicino ai 100 °C e il core del nodo stesso in throttling. **3,5 GHz** è la frequenza più alta che la macchina regge per un run intero, ed è la frequenza di default del banco.

> La frequenza pesa quasi in proporzione, ma non del tutto, perché la cache condivisa e la memoria non accelerano con il core. Le cifre assolute si citano con la loro frequenza, e con il tipo di core (sezione 9.11).

10.5 Il risultato: misure che si ripetono

che cosa si ripete	variazione
i tre giri di una misura ((massimo − minimo) / mediana)	1,3 % in mediana
il nodo senza rete, in 109 sessioni diverse (P-core)	1,1 % in mediana, al più 5 %
stesso modello e stessa versione, due sessioni (30 coppie)	1,8 % in mediana, al più 7 %

Una differenza sotto il 5 % fra misure di sessioni diverse non è una differenza. E i pacchetti respinti con il nodo sotto capacità sono **zero**: il core del nodo non dorme, e nessun altro lavoro gli viene dato.

10.6 Come si rifà tutto

Due script rifanno ogni misura di questo quaderno, nelle stesse condizioni:

```
bash ipa/test/run_kernel_checks.sh        # tutto cio' che non usa traffico vero (~20 min)
bash ipa/test/run_population_campaign.sh  # la campagna sulla popolazione (~4 h 30)
```

Il primo scrive CSV e registri in `results/kernel_<data>/` e i grafici in `docs/figures/`; il secondo scrive in `results/popolazione_<data>/`, con le condizioni della macchina accanto a ogni misura e la sintesi in `sintesi.md`.

CAPITOLO 11

Altri modelli, altre reti

11.1 La domanda

Quasi ogni numero dei capitoli precedenti viene da un modello solo, il 65-4-4-7 addestrato su Germany50. Un banco che funziona solo su quel modello potrebbe avere la sua forma scritta dentro senza che nessuno se ne accorga: gli strati di P3 elencati a mano, sei bit di stato dei link, il significato delle classi letto sempre dalla scheda del checkpoint. Un modello diverso girerebbe allora con i pesi, o con le classi, di un altro. L'obiettivo di questo capitolo: **lo stesso test, per qualsiasi modello e qualsiasi rete**, e un errore chiaro quando la combinazione non ha senso.

11.2 Scenario e modello

Le due parole hanno un significato preciso.

- **Lo scenario è la rete**: quattro numeri in `topologies/<nome>/topology_config.json`, cioè quante interfacce ha al massimo un nodo, quanti nodi ha la rete, quante code ha ogni interfaccia e con che TTL i pacchetti ci entrano.
- **Il modello è la rete neurale**: pesi, forma, ingressi (ognuno con la sua larghezza e la sua scala) e significato di ogni classe.

Scenario	interf.	nodi	code	TTL	modelli che ci girano
`germany50`	6	52	4	30	checkpoint, ipa_like, deep, sparse, sparse_hetero_11
`germany50_ttl16`	6	52	4	16	ipa_ttl16
`synth_small`	3	8	–	16	small
`synth_ones`	3	4	–	8	ones
`synth_large`	8	100	–	64	large
`synth_mixed`	4	12	1	30	mixed

Oltre a Germany50 ci sono le reti dei modelli sintetici. Non sono topologie reali: sono le dimensioni per cui ciascun modello sintetico è stato generato.

11.3 Quando un modello può girare su una rete

Ogni ingresso del modello prende la sua larghezza da una dimensione della rete: lo stato dei link e la porta d'ingresso dalle interfacce, l'identità del nodo dai nodi, l'occupazione delle code dalle code. E il TTL il modello lo legge diviso per il TTL iniziale con cui è stato addestrato (sezione 2.3 nel capitolo sulla matematica). Quindi:

> Un modello gira su una rete solo se **ogni suo ingresso è largo quanto la dimensione della rete da cui dipende** e se **la scala del suo TTL è il TTL iniziale della rete**.

Esempio: il checkpoint ha il one-hot del nodo largo 52. Su una rete da 30 nodi metà delle sue colonne non corrisponderebbe a nessun nodo, e il test si ferma subito con

```
nodi: il modello ne vuole 52, la rete ne ha 30
```

Esempio con il TTL: `ipa_ttl16` ha la stessa forma del checkpoint, ma è addestrato con il TTL diviso per 16. Su Germany50, dove i pacchetti partono da 30, vedrebbe valori fino a quasi 2 invece che fino a 1, fuori da tutto ciò su cui è stato addestrato. Per questo gira sulla sua rete, `germany50_ttl16`.

C'è poi un secondo filtro, fra modello e *pipeline*. Ogni pipeline ha dei tetti fissati quando il programma viene compilato:

- **P2**: al massimo 8 neuroni per strato, strati dal terzo in poi larghi come il secondo, 1 024 pesi in tutto;
- **P3**: al massimo 8 neuroni per strato, 16 strati, 2 048 pesi;
- **P1**: nessuno, perché il programma si genera per quel modello.

Il modello `large` (117-8-8-9) ha 1 097 pesi e un primo strato da 117 per 8: P2 e P3 non lo possono caricare. Non è un errore del test: la combinazione è **non applicabile**, ed è contata a parte.

11.4 Come è costruito

Quattro pezzi, ognuno con un compito solo.

1. `model_source.py` **carica qualsiasi modello nello stesso formato**: il checkpoint, un altro `.pt`, un preset sintetico (`synth:deep`) o una cartella. Il significato delle classi è quello che il modello dichiara. Contiene anche il controllo modello contro rete.
2. `pipeline_limits.py` **dice se una pipeline regge una forma**, con i numeri presi dai moduli che compilano i programmi, così i due non possono divergere.
3. `pipeline_setup.py` **carica un modello in una pipeline**: baseline, P1, P1.5, P2, P3. Sono gli stessi costruttori del confronto a tre vie.
4. `model_under_test.py` **legge** `-model` **e** `-topology` e li passa ai processi figli con due variabili d'ambiente.

Senza `-model` si usa il checkpoint; con `-model checkpoint` si usa lo stesso checkpoint per la strada generica, e il risultato è identico (sezione 11.6).

```
sudo python3 ipa/test/test_weights_and_metrics.py --only kernel --model synth:deep
sudo python3 ipa/test/test_forwarding.py --model synth:small
sudo python3 ipa/test/bench_capacity.py --mode compare --generator xdp --model synth:deep
sudo python3 ipa/test/bench_capacity.py --mode compare --generator xdp \
     --model ipa/synth/population/m000_ipa17_h2_o3
```

Senza `-topology` viene scelta la rete compatibile (Germany50 se lo è).

11.5 Che cosa controllano i test

Passare un modello diverso chiede controlli precisi in tre punti.

L'azione, non solo la classe. Col checkpoint, fra TTL 2 e 6, la classe è sempre un inoltro; un altro modello può decidere DROP proprio lì. Il test controlla che il programma abbia scelto la classe del riferimento *e* ne abbia eseguito l'azione: redirect per un inoltro, `XDP_DROP` per uno scarto, `XDP_PASS` per una classe inutilizzata.

Tutte le classi che il modello sa decidere. Il fabric cerca un caso per ogni classe cambiando stato dei link e TTL; per le classi che così non trova cambia anche porta d'ingresso,

nodo e code, e ogni caso scrive i suoi valori nelle mappe prima di spedire il pacchetto. Col checkpoint basta lo stato dei link; coi pesi casuali dei sintetici conta anche il resto. Classi consegnate:

Modello	classi consegnate
checkpoint	0, 1, 2, 3, 4, 5
ipa_like, ipa_ttl16	2, 3, 4, 5
small, mixed	0, 1, 2, 3
large	1, 4, 6, 8
deep / sparse / ones	0, 1, 4 / 1, 5 / 0

Le classi che mancano il modello non le decide mai: su 200 000 ingressi casuali `deep` sceglie solo 0, 1 e 4, e `ones`, che ha tutti i pesi uguali a 1, sceglie sempre la classe 0. È una proprietà del modello, non un limite del test.

Il traffico misura un inoltro. I generatori mandano pacchetti con TTL 32. Col checkpoint, con tutti i link attivi, quel pacchetto viene inoltrato; con `large` invece viene scartato, e il banco misurerebbe lo scarto chiamandolo inoltro. Il banco allora cerca lo stato dei link che porta a un inoltro, lo scrive nella mappa e lo stampa:

```
traffico di large con link_state=00111000, TTL 32 -> classe 4 (FORWARD)
```

Se nessuno stato dei link porta a un inoltro, la pipeline non si misura. Il generatore della popolazione fa lo stesso controllo: un seme il cui modello scarta il pacchetto di prova in ogni stato dei link si scarta, e si passa al successivo (capitolo 12).

11.6 La prova che la strada generica non cambia niente

Il checkpoint passato esplicitamente, `-model checkpoint`, va per la strada generica. Confrontato con il checkpoint senza `-model`:

- pesi, scala e significato delle classi identici; il riferimento Python dà la stessa classe e lo stesso logit su 3 000 ingressi;
- il sorgente di P1 e la foglia di P2 sono identici *byte per byte*;
- il riferimento generico coincide con quello scritto per il 65-4-4-7 su 20 000 ingressi.

E la correttezza sui modelli nuovi:

- nel kernel, ogni scenario per ogni modello compatibile per ogni pipeline, 40 ingressi a testa, sotto due identificativi di modello: **110 su 110**, più `large` non applicabile a P2 e P3;
- suite kernel su nove modelli: tutto PASS.

Quanto costano modelli diversi non lo dicono questi nove modelli, ma la popolazione di 109 del capitolo 12, misurata con questa stessa strada generica.

11.7 Che cosa non fa

- **Nessun modello sintetico è addestrato**: servono a provare le pipeline su forme diverse, non a dire qualcosa sull'instradamento.

CAPITOLO 12

Nessun modello privilegiato: la popolazione

12.1 La domanda

Una misura su un modello dice quanto costa *quel* modello. La domanda della tesi è un'altra: quanto costa ciascuna delle quattro pipeline, con qualunque modello ragionevole le si dia. Per rispondere, ogni banco gira qui su una **popolazione di 109 modelli sintetici**, e ogni conclusione ha una di queste quattro forme:

- un **intervallo**: la mediana sulla popolazione e, fra parentesi quadre, il minimo e il massimo, dal modello più leggero al più pesante;
- un **conteggio**: su quanti modelli un'affermazione regge ("103 modelli su 103");
- una **legge**: costo $= a + b \times$ MAC eseguite $+ c \times$ strati, stimata con i minimi quadrati su tutti i modelli densi, con l'errore che fa su modelli lasciati fuori dalla stima;
- un **rapporto modello per modello**: per esempio la capacità sull'E-core divisa quella sul P-core dello stesso modello.

> In questo capitolo nessuna cifra è di un modello solo. Le cifre vengono da `results/popolazione_2026-10/`, e le tira `report_population.py`.

12.2 La popolazione

La genera `ipa/test/model_population.py` (tier `base`), con un seme fisso: rigenerata, è identica byte per byte. Ogni modello ha il suo seme, e un seme si scarta se il modello non inoltra il pacchetto di prova o non raggiunge almeno due classi.

Famiglia	modelli	che cosa varia
base	53	7 composizioni dell'ingresso × profondità 1, 2, 3, 4, 6 × larghezza 2, 4, 8 (metà delle combinazioni), 3–7 uscite
replica	20	6 forme, 3 semi in più ciascuna (uno con pesi a distribuzione normale): quanto il costo dipende dai pesi estratti
sparsità	8	4 forme con il 50 e il 90 % dei pesi esattamente a zero
imbuto	10	5 forme con circa 592 pesi distribuiti su 1–5 strati, su 2 composizioni
strati	18	1–6 strati nascosti con 102–113 MAC eseguite, su 3 composizioni: il costo di uno strato a moltiplicazioni ferme (sezione 12.11)
totale	109	5–117 ingressi, 1–6 strati, 2–8 neuroni per strato, 21–1 349 pesi, 16–480 MAC eseguite

Le sette composizioni dell'ingresso mettono insieme in modi diversi le feature dense (stato dei link, occupazione delle code, TTL) e le one-hot (interfaccia d'ingresso, identità del nodo): `dense5` e `dense17` solo dense, `onehot105` quasi solo one-hot, `ipa17`, `ipa65`, `ipa117` e `mixed33` miste, con reti da 10, 24, 52 e 100 nodi.

La famiglia *strati* sta in fondo al manifesto (`m091`–`m108`): entra in ogni intervallo, ha una sezione sua, e resta fuori dalla stima della legge di costo, che la usa come prova. Sei modelli superano il tetto sui pesi di P2 (`MAX_WEIGHT_ENTRIES`): restano nella popolazione, e per P2 le cifre sono su 103 modelli. Le altre pipeline li eseguono tutti.

I test lunghi girano su sottoinsiemi scelti dal modello più leggero al più pesante: 6 modelli per le curve a traffico crescente e per le taglie del pacchetto, 14 per il LP E-core, 8 per i due core e per le classi, 12 per i TTL misti.

12.3 Come si misura, e quanto è pulito il dato

Tutto lo fa `ipa/test/run_population_campaign.sh`, con il disegno del capitolo 9: cambia solo il core del nodo (P-core cpu6, E-core cpu12, LP E-core cpu20), il generatore e l'uscita restano su P-core, i core del banco a 3,5 GHz (2,5 il LP E-core).

Sezione	che cosa misura
`corretto`	riferimento intero contro eBPF, ogni modello e ogni pipeline (`BPF_PROG_TEST_RUN`, 200 ingressi)
`rif`	rxonly e baseline senza modello, una volta per tipo di core
`pop`	ogni modello su P-core ed E-core: capacità, tempo di CPU, istruzioni e cicli per pacchetto, programmi caricati; e $T_2 - T_1$, $T_3 - T_2$ a 0,25 e 0,5 milioni di pacchetti al secondo
`lpe`, `carico`, `core2`	i sottoinsiemi: LP E-core, traffico crescente, due core
`classi`, `mix`, `taglie`	una sessione per classe, TTL diversi a ogni pacchetto, 64/512/1 514 byte
`solo`	il programma da solo, le letture di tabella e i salti per pacchetto, installare un modello (sezione 12.12)

La campagna dura 3 h 54 min e riprende da dove si è fermata: una misura il cui CSV c'è già si salta. Ogni misura scrive accanto ai numeri le condizioni della macchina, e `report_population.py` lascia fuori da ogni intervallo e da ogni legge quelle in cui la macchina non è rimasta nelle condizioni dichiarate o che il controllo di validità del banco ha bocciato; le elenca in `escluse.csv`, e `POP_RIFAI_DISTURBATE=1` le rifà.

Il throttling del portatile

Con l'alimentatore collegato e a 57–72 °C, su questo portatile un evento di throttling di pacchetto arriva circa una volta al minuto, comunque si usi la macchina: lo decide il firmware. Una misura a massima spinta dura circa 30 s, una curva a traffico crescente circa 160 s: rifarle finché non lo prendono non converge. Un evento però tocca uno o due giri, non la misura. Il report toglie allora i soli giri la cui finestra cade entro 1 s da un evento (il registro della macchina campiona al secondo) e rifà le mediane con la stessa funzione del banco, purché ogni pipeline, o ogni punto della curva, conservi almeno 2 giri su 3. Se il disturbo non è solo throttling (powerclamp, batteria, frequenza fuori target) la misura resta fuori.

Alla fine: **2 misure fuori su 580**, e 12 tenute senza i giri colpiti (26 giri o finestre su 1 980).

12.4 Correttezza

Su ogni modello il riferimento intero e il programma eBPF decidono la stessa classe, pacchetto per pacchetto: **P1 su 109 modelli su 109, P2 su 103 su 103, P3 su 109 su 109**. I 6 modelli fuori da P2 sono quelli oltre il suo tetto sui pesi: un limite della pipeline, non un errore.

12.5 Il costo del nodo

	baseline	P1 statica	P1.5	P2	P3
P-core, 64 byte, nodo saturo: mediana [minimo–massimo]					
ns di CPU per pacchetto	66	119	124	292	487
	[63–69]	[85–221]	[86–222]	[196–512]	[307–918]
costo della rete (ns)	—	53	58	226	421
		[19–155]	[19–156]	[129–446]	[240–852]
milioni di pacchetti al secondo	15,0	8,4	8,0	3,4	2,1
	[14,5–15,4]	[4,5–11,7]	[4,5–11,6]	[2,0–5,1]	[1,1–3,3]
istruzioni eseguite per pacchetto	606	1 168	1 349	4 578	7 620
istruzioni caricate	135	650	961	16 813	12 288
istruzioni per ciclo	2,6	2,9	3,2	4,5	4,5

109 modelli (P2: 103). Ricevere costa 40 ns (rxonly), leggere l'intestazione e inoltrare altri 27: la baseline, uguale con ogni modello.

Modello per modello, sopra la baseline:

- **P1 $\leq$ P2 $\leq$ P3 su 103 modelli su 103**;
- rendere configurabile la forma (P2) costa **4,4 [2,7–12,8] volte** P1, anche la profondità (P3) **7,8 [4,9–24,6] volte**; P3 costa 1,9 [1,4–2,3] volte P2; P1.5 costa 1,1 [0,9–1,7] volte P1. I rapporti più alti vengono dai modelli più piccoli, dove P1 costa poche decine di ns;
- P2 carica più istruzioni di P3 ma ne esegue meno su 103 modelli su 103: la dimensione del programma non ordina le pipeline per lavoro. Dentro una pipeline la dimensione ordina bene i modelli per P1 (correlazione di rango 0,98), meno per P1.5 (0,90) e P2 (0,79); P3 ha la stessa dimensione, 12 288 istruzioni, con ogni modello.

12.6 P-core, E-core, LP E-core

E-core rispetto al P-core	baseline	P1 statica	P1.5	P2	P3
capacità E/P	0,63	0,76	0,75	0,82	0,83
	[0,61–0,66]	[0,69–0,91]	[0,68–0,88]	[0,77–0,87]	[0,72–0,89]
istruzioni per ciclo E/P	0,65	0,78	0,77	0,84	0,84
cicli della rete E/P	—	0,95	1,01	1,09	1,13
		[0,55–1,20]	[0,59–1,34]	[1,02–1,18]	[1,05–1,40]
LP E-core rispetto al P-core					
capacità LP/P (14 modelli)	0,26	0,35	0,35	0,45	0,48
		[0,32–0,45]	[0,32–0,44]	[0,42–0,48]	[0,45–0,53]

- **Le istruzioni per pacchetto non dipendono dal core**: fra P-core ed E-core differiscono al più dello 0,3 % su 108 modelli, sul LP E-core al più dello 0,7 % sulle pipeline.
- **L'E-core rallenta il nodo, non la rete**: la parte fissa (i cicli della baseline) passa da 231 a 360 cicli, 1,56 [1,50–1,61] volte, su ogni modello. I cicli della sola rete cambiano poco. Per

questo la capacità E/P sale con il peso della pipeline: più tempo nella rete, meno conta il core.

- **Il LP E-core paga anche la memoria**: sta fuori dalla L3 e ha 13–21 mancate dell'ultimo livello di cache per pacchetto. A 2,5 GHz regge dal 32 al 53 % del P-core sulle pipeline.

12.7 Sotto traffico crescente, e con due core

Le curve a traffico crescente (da 0,5 a 12 milioni di pacchetti al secondo, sei modelli, P-core ed E-core) salgono lungo la diagonale fino alla capacità e poi restano piatte. Il massimo inoltrato sta a **0,98 [0,88–1,03]** della capacità a massima spinta sul P-core e a **0,99 [0,94–1,03]** sull'E-core, su 24 curve per core. Della perdita oltre la capacità, la quota persa alla coda d'ingresso, prima del programma, è 1,000 in mediana: il collo di bottiglia è il programma eBPF.

L'eccezione

P1 sui modelli più leggeri regge oltre 9 milioni di pacchetti al secondo. Lì comincia a perdere anche l'uscita: la quota persa alla coda scende fino a 0,69, il massimo inoltrato fino a 0,88 della capacità. È il limite del cavo virtuale e del core d'uscita, non della rete neurale.

Con **due core** del nodo, ciascuno con la sua coda e il suo core d'uscita, la capacità è il **99 [94–103] %** del doppio sul P-core e il **99 [98–101] %** sull'E-core, su 40 coppie modello-pipeline per core. Un core e due core sono misure di sessioni diverse: la stessa misura rifatta in un'altra sessione cambia dell'1,8 % in mediana e fino al 5 % (90° percentile), quindi i valori sopra il 100 % sono rumore, e il risultato è *il doppio, entro il rumore*.

12.8 Il programma, il trasporto, e due modi di misurare la rete

Con i tre orologi (T_1 all'ingresso, T_2 prima di inoltrare, T_3 all'arrivo al nodo successivo), sui 91 modelli che li misurano (la famiglia *strati* è misurata senza):

- **il trasporto** $T_3 - T_2$ (redirect, cavo virtuale, ricezione) è 187–221 ns sul P-core e 226–274 sull'E-core, su ogni modello; fra la pipeline più cara e la meno cara dello stesso modello differisce del 4 [1–17] %. È il minimo: la media contiene l'attesa in coda, perché qui l'uscita gira sul core del nodo, e cresce col traffico (da circa 550 a circa 1 200 ns fra 0,25 e 0,5 milioni di pacchetti al secondo);
- **il programma** $T_2 - T_1$ (minimo) va da 27 ns per la baseline a 391 per P3 in mediana, e per P3 da 237 a 797 ns secondo il modello.

Il costo della rete si ricava in due modi indipendenti: tempo di CPU a nodo saturo meno la baseline, oppure media di $T_2 - T_1$ a traffico leggero meno la baseline. Il loro rapporto, modello per modello:

traffico leggero / nodo saturo	P1 statica	P1.5	P2	P3
P-core	0,77 [0,48–1,02]	0,83 [0,50–1,02]	1,03 [0,94–1,09]	1,02 [0,94–1,11]
E-core	1,26 [0,34–2,73]	1,22 [0,30–2,52]	1,08 [0,93–1,35]	1,07 [0,82–1,13]

Per **P2 e P3 le due misure coincidono** su ogni modello. Per P1 e P1.5 no: la loro rete costa poche decine di ns sopra una baseline di circa 66, e la differenza fra due misure gran-

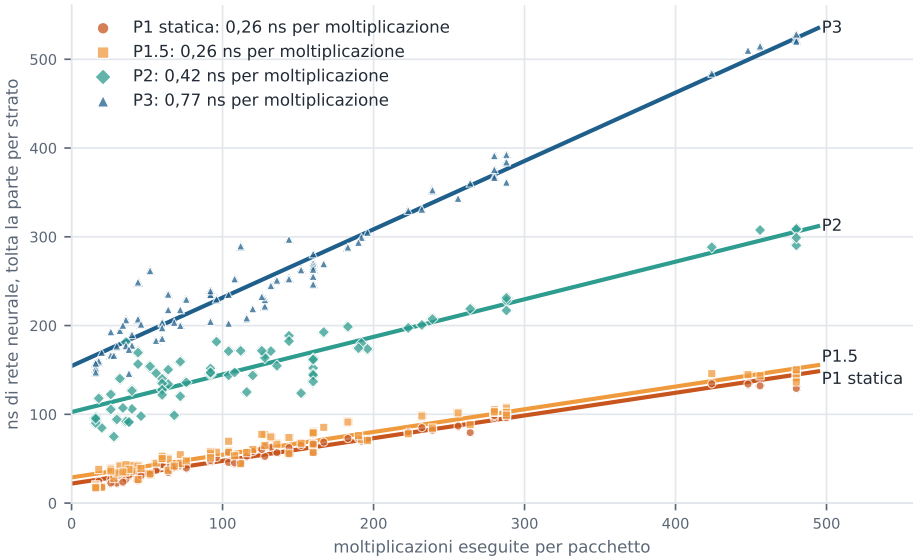
di è rumorosa. Il loro costo della rete si cita con il metodo che lo ha misurato; quello di riferimento è il tempo di CPU a nodo saturo, il costo che il nodo paga davvero.

12.9 La legge di costo

La legge è stimata sugli 83 modelli densi (famiglie base, replica e imbuto; 77 per P2), sul P-core:

	a (ns)	b (ns/MAC)	c (ns/strato)	r^2	errore su modelli non visti
P1 statica	22	0,26	1	0,98	6 %
P1.5	29	0,26	1	0,95	9 %
P2	103	0,42	20	0,93	6 %
P3	154	0,77	46	0,99	3 %

Errore = errore relativo mediano su un modello lasciato fuori dalla stima, uno alla volta. Sull'E-core P3 vale 153 ns + 0,79 ns/MAC + 62 ns/strato.



Ogni punto è un modello denso: costo della rete meno $c\times$ strati, contro le MAC eseguite. Le rette sono $a + b\times$ MAC.

Il trucco è contare le moltiplicazioni **davvero eseguite**. Una feature one-hot occupa molte colonne nella matrice dei pesi ma ne attiva una sola. Contando le MAC dalla forma del modello la legge spiega meno (r^2 0,58–0,79); contando quelle eseguite spiega il costo di tutte e quattro le pipeline, e prevede un modello nuovo con qualche punto percentuale di errore.

Dove va il tempo

La legge dice anche *dove* va il tempo. Per ogni modello denso si divide il costo previsto nei suoi tre termini; la tabella dà la quota mediana di ciascuno.

	fisso (a)	moltiplicazioni ($b\times$ MAC)	strati ($c\times$ strati)
P1 statica	40 % [14–77]	50 % [14–80]	8 % [3–20]
P1.5	48 % [18–83]	46 % [12–78]	5 % [2–14]
P2	47 % [23–69]	18 % [5–46]	31 % [18–54]
P3	37 % [18–60]	18 % [5–44]	39 % [23–64]

Con i pesi nel codice (P1, P1.5) circa metà del tempo è aritmetica. Con i pesi in tabella (P2, P3) circa l'82 % è **struttura**: il termine fisso (dispatcher, lettura del descrittore del modello, costruzione degli ingressi da tabella) e quello per strato (la tail call allo strato generico e la rilettura del suo contesto). La legge è lineare proprio per questo: ogni tail call e ogni lettura di tabella costa quasi sempre lo stesso, e quante ce ne sono dipende solo dalla forma del modello. Il termine per strato è una media: che cosa ci sta dentro, strato per strato, lo dice la sezione 12.11.

Anche il termine per moltiplicazione contiene le tabelle: P3 paga 0,77 ns per MAC contro 0,26 di P1 perché legge ogni peso dalla memoria di una mappa, in un ciclo non srotolato, mentre in P1 il peso è una costante nel codice. Nessun pezzo del costo per pacchetto passa dallo user space: le mappe si leggono dentro il kernel (`bpf_map_lookup_elem`, che per le mappe array il verifier di solito sostituisce con un calcolo di indirizzo), e lo user space le scrive solo quando installa un modello. Le letture e i salti si contano (sezione 12.12): P3 fa 36 [23–54] letture di tabella per pacchetto, circa 6 in più per strato, e un salto per strato. Quanto dei 46 ns per strato sia il salto e quanto le letture, la campagna non lo separa.

La stessa forma spiega le istruzioni: P3 esegue **901 istruzioni in più per ogni strato** (r^2 0,99), identiche su P-core ed E-core, P2 370 (sezione 12.11). Le istruzioni caricate: P2 ne aggiunge 887 per strato (r^2 0,96), perché ricompila e srotola per ogni profondità; P3 ne ha 12 288 con ogni modello.

12.10 La forma del modello

- **Profondità.** A MAC ferme uno strato in più costa circa 46 ns a P3, 20 a P2, 1 a P1 e P1.5 (la legge). P2 lo paga anche in spazio, con un programma più grande da ricompilare.
- **A parità di pesi** (famiglia imbuto, 592 pesi da 1 a 5 strati), da 1 a 5 strati la rete costa il 48 [44–53] % in più a P2 e il 63 [55–70] % in più a P3, circa 27 e 58 ns per strato; a P1 e P1.5 il 14–17 %, circa 4 ns per strato. Con le pipeline generiche conviene una rete larga e poco profonda; con le specializzate la forma si sceglie per l'accuratezza.
- **Larghezza.** Da 2 a 8 neuroni per strato la rete sale su tutte (in mediana P1 da 31 a 84 ns, P3 da 347 a 516): un neurone in più è lavoro a ogni pacchetto. La legge lo conta nelle MAC.
- **Taglia della rete.** Sui 56 modelli con l'identità del nodo in ingresso (reti da 10 a 100 nodi), lo scarto dalla legge non cresce con i nodi: al più 0,03 ns per nodo, su tutte le pipeline. La one-hot del nodo attiva una colonna sola, qualunque sia la sua larghezza.
- **Pesi a zero.** Al 90 % di zeri P1 costa 0,44 [0,31–0,63] volte quanto la legge densa prevede per la stessa forma, P1.5 0,62 [0,27–0,83], e il programma di P1.5 scende a 0,34–0,35 volte quello del modello denso: il compilatore cancella le moltiplicazioni per zero. P2 e P3 restano sulla legge (1,02 e 1,01) con un programma identico.
- **I pesi estratti.** Sei forme con quattro semi ciascuna: il costo della rete varia al più del 5–6 % per P1, P1.5 e P2 e del 2 % per P3 (coefficiente di variazione fra i semi).

12.11 Il costo di uno strato, a moltiplicazioni ferme

La legge separa strati e moltiplicazioni con una regressione, su modelli in cui le due cose crescono insieme. Per misurare uno strato direttamente serve cambiare il numero di strati *senza* cambiare le moltiplicazioni: è la famiglia *strati*. Per tre composizioni dell'ingresso (`dense17`, `mixed33`, `ipa65`) una rete con 1, 2, 3, 4, 5 e 6 strati nascosti, sempre con 102–113 MAC eseguite, e con le forme che P2 sa eseguire (dal terzo strato in poi larghi come il secondo, al più 8 neuroni). Per esempio `17-5-5` (1 strato, 110 MAC) e `17-3-3-3-3-3-5` (6 strati, 111 MAC).

P3, istruzioni per pacchetto	1	2	3	4	5	6	per strato
`dense17`	6 375	7 057	7 888	9 087	9 971	10 933	912
`mixed33`	5 718	6 536	7 696	8 713	9 252	9 644	785
`ipa65`	5 479	6 342	7 517	7 784	8 616	9 962	897

Istruzioni eseguite per pacchetto, misure grezze (identiche su P-core ed E-core). Per strato: (6 strati − 1 strato) / 5.

In media P3 sale di circa 900 istruzioni per strato e P2 di circa 370 (270–450 secondo la composizione): gli stessi valori della legge, stimata su altri modelli. P1 e P1.5 restano piatte, fra 1 150 e 1 400. Ma i gradini non sono uguali: dentro la stessa composizione vanno da +270 a +1 350 istruzioni. Il costo di uno strato dipende dalla sua forma, e il codice dice come.

- **P3.** Ogni strato nascosto è un programma (`EBPF_LAYER_HIDDEN`) raggiunto con un tail call: rilegge `scratch_meta`, `layer_registry` e `layer_shapes` (due tabelle hash), poi i pesi e le attivazioni. La forma la scopre a ogni pacchetto, quindi i cicli non possono avere la sua larghezza: sono srotolati a `MLH_MAX_H` = 8, perché il verifier vuole un numero di giri costante. Ogni neurone d'uscita esegue 8 controlli `i >= n_in`, e i neuroni oltre `n_out` girano a vuoto.
- **P2.** La profondità è fissata alla compilazione (`build_arch_leaf`): gli strati sono blocchi in fila nello stesso programma, senza salti né riletture. La larghezza no: i cicli sono srotolati a 8 e quello interno non ha il controllo, quindi ogni neurone fa sempre 8 moltiplicazioni, per zero oltre la larghezza vera.
- In entrambe il **primo strato** è srotolato alla misura massima: il suo costo dipende dalla composizione dell'ingresso, non dalle MAC eseguite.

Scritta con questi termini, la legge delle istruzioni ha un costo fisso per composizione, uno per strato, e uno per neurone e per moltiplicazione degli strati dopo il primo. Stimata su tutti i modelli densi (101; 95 per P2):

istruzioni per	P3	P2
strato	687	39
neurone dopo il primo strato	44	122
moltiplicazione dopo il primo strato	14	∼0
errore su un modello lasciato fuori	0,1 %	1,0 %

In P3 uno strato costa circa **690 istruzioni fisse** (salto, letture di tabella, cicli a vuoto) **più circa 44 per ogni suo neurone**: i 901 della legge sono la media sulle forme della popolazione. In P2 lo strato in sé costa poco, perché non salta: paga i neuroni, ognuno con 8 moltiplicazioni piene. È il prezzo della genericità sulla larghezza: un programma che vale per ogni modello fino a 8 neuroni lavora sempre come se lo strato prima fosse largo 8.

Un gradino piccolo, con `ipa65`: da `6-4-4` a `8-2-2-2` P3 sale di 267 istruzioni. Lo strato in più vale +687, ma dopo il primo strato ci sono 24 moltiplicazioni in meno (-336) e 2 neuroni in meno (-88), e il primo strato passa da 6 a 8 neuroni (+26): +292 previsti.

La legge in tre termini ($a + b\cdot$ MAC $+c\cdot$ strati) resta quella che si usa per stimare il tempo, perché basta la forma del modello. Sui 18 modelli di questa famiglia, che non usa per la stima, sbaglia il tempo sul P-core del 5 % per P2 e P3, del 6 % per P1 e del 13 % per P1.5: un poco più del 3–9 % stimato lasciando fuori un modello alla volta.

12.12 Il programma da solo, e il cambio di modello

La sezione `solo` della campagna (`bench_program_alone.py`) carica ogni modello in ogni pipeline con lo stesso costruttore dei banchi sotto traffico, e lo esegue con `BPF_PROG_TEST_RUN` sul pacchetto e con gli ingressi del traffico: TTL 32, il nodo e la porta del fabric, lo stato dei link che fa inoltrare il modello. Ogni misura ha inoltrato il pacchetto in tutte le esecuzioni; una misura è fuori perché la macchina era disturbata.

	baseline	P1 statica	P1.5	P2	P3
letture di tabella per pacchetto	3	5 [4–5]	6 [5–6]	12 [11–12]	36 [23–54]
salti fra programmi per pacchetto	0	1	1	1	1 per strato
installare un modello (ms)	—	0,6 [0,3–1,5]	0,9 [0,3–5,6]	0,3 [0,2–0,4]	0,3 [0,2–0,5]
generare e compilare P1 (ms)	—	50 [34–114]	61 [34–181]	—	—
compilare il programma generico (s)	—	—	—	1,6 [1,4–1,7]	1,2 [1,2–1,2]

Mediana [minimo–massimo] su 108 modelli (P2: 102). Installare = un modello nuovo su un nodo in servizio: per P1 caricare l'oggetto compilato (verifier e JIT), per P2 e P3 scrivere le tabelle. Generare e compilare P1 si fa fuori dal nodo, con clang senza cache, su 109 modelli.

- **La configurabilità si conta.** Le letture di tabella per pacchetto crescono da P1 a P3, e per P3 crescono con la profondità: circa 6 letture in più per strato. I salti fra programmi di P3 sono uno per strato su 108 modelli su 108. È la struttura che la legge di costo vede nel termine per strato.
- **Cambiare modello costa millisecondi o meno a tutte**, su ogni modello: sul nodo al più 6 ms. La differenza fra le pipeline è dove serve un compilatore: P1 ne vuole uno per ogni modello nuovo, fuori dal nodo (34–181 ms); P2 e P3 compilano il programma generico una volta sola, all'avvio.
- **Il programma da solo, a chiamate singole**, costa per P2 e P3 quanto sotto traffico o poco di più: la rete a nodo saturo sta a 0,94 [0,77–1,10] e 0,91 [0,84–0,97] volte la rete da sola. Ogni esecuzione è una chiamata al kernel, che raffredda cache e predittori come fa il traffico.
- **Il programma da solo, in un ciclo caldo** (blocchi di esecuzioni in cui il TTL che scende non cambia la classe decisa): coincide con il pacchetto più veloce a traffico leggero (da

0,95 a 1,04 volte), e il nodo saturo costa 1,08 (P2) e 1,11 (P3) volte il ciclo caldo, 1,28–1,33 volte per P1.5 e P1, la cui rete è piccola rispetto a quello che il traffico sposta in cache.

12.13 Il resto è solido

Sui sottoinsiemi, dal modello più leggero al più pesante:

- **taglia del pacchetto**: fra 64 e 1 514 byte il tempo del nodo cambia di 1,2 [0,3–3,1] % (30 coppie modello-pipeline);
- **porta d'uscita**: le classi che inoltrano costano uguale, 0 [0–9] % fra la più cara e la meno cara; scartare costa 51 [17–63] ns in più, perché la memoria del pacchetto si libera sul core del nodo (un effetto del banco);
- **TTL diversi a ogni pacchetto**: 0 [−6–4] % sul tempo del nodo rispetto al pacchetto fisso (48 coppie).

12.14 Le affermazioni, e su che cosa reggono

Affermazione	sulla popolazione
Tutte le pipeline decidono come il riferimento intero	P1 109/109, P2 103/103, P3 109/109
P2 ha un tetto sui pesi che le altre non hanno	6 modelli su 109 fuori da P2
P1 $\leq$ P2 $\leq$ P3 nel costo della rete	103/103
Rendere configurabile la forma costa più volte P1	P2/P1 4,4 [2,7–12,8], P3/P1 7,8 [4,9–24,6]
P2 è il programma più grande ma P3 esegue di più	103/103
Le istruzioni per pacchetto non dipendono dal core	entro 0,3 % (P/E), 0,7 % (LP)
L'E-core rallenta soprattutto la parte fissa	cicli fissi E/P 1,56 [1,50–1,61]
Il LP E-core regge un terzo–metà del P-core	0,32–0,53 sulle pipeline
Oltre la capacità si perde alla coda d'ingresso	quota 1,000 [0,689–1,000]
Il ginocchio coincide con la capacità	0,98 [0,88–1,03] (P), 0,99 [0,94–1,03] (E)
Due core reggono il doppio	99 [94–103] % (P), 99 [98–101] % (E)
Il trasporto è un costo comune	187–221 ns (P), 226–274 (E)
Traffico leggero e nodo saturo danno lo stesso costo (P2, P3)	1,03 e 1,02 [0,94–1,11]
Il costo segue a + b $\times$ MAC eseguite + c $\times$ strati	r^2 0,93–0,99, errore 3–9 %
Con i pesi in tabella pesa la struttura, non l'aritmetica	P2, P3: $\sim$82 % fisso + strati
Le MAC eseguite spiegano meglio di quelle della forma	4 pipeline su 4
Ogni strato di P3 costa le stesse istruzioni	+901 per strato, r^2 0,99
A pari pesi la profondità si paga solo con P2 e P3	+48 % e +63 % da 1 a 5 strati
La taglia della rete non costa	$\leq$ 0,03 ns per nodo
I pesi a zero aiutano solo P1 e P1.5	0,44 e 0,62 del previsto al 90 %
I pesi estratti contano poco	variazione $\leq$ 6 % (P3 2 %)
Le letture di tabella crescono da P1 a P3, e con gli strati	5, 6, 12, 36 [23–54]; +6 per strato
P3 salta una volta per strato	108/108
Sotto carico P2 e P3 costano poco più del ciclo caldo	1,08 e 1,11 volte
Cambiare modello costa millisecondi o meno sul nodo	$\leq$ 6 ms; P1 compila fuori in 34–181 ms
Taglia del pacchetto, porta, TTL non contano	1,2 %, 0 %, 0 % in mediana

12.15 Che cosa non dice

- **I modelli sono sintetici**: coprono forme, composizioni e pesi, non l'accuratezza di un modello addestrato.
- **Il ciclo caldo vale solo dove il TTL lo permette**: si misura sui modelli in cui almeno 10 esecuzioni di fila decidono la stessa classe.
- **Le letture di tabella e i salti si contano, ma non si separano nel tempo**: quanto dei circa 46 ns per strato di P3 sia il salto e quanto le letture, la campagna non lo dice.
- **Il banco resta un portatile con cavi virtuali**: nessuna scheda di rete, generatore sulla stessa macchina. I costi del programma sono quelli veri; quello di ricezione su una scheda vera può essere diverso.

12.16 Come rifarla

```
cd ~/Scrivania/ipa_lab
CAMPAGNA=results/popolazione_2026-10 bash ipa/test/run_population_campaign.sh
# prova breve, due modelli in tutte le sezioni
CAMPAGNA=results/prova_pop POP_SOLO="m000 m037" bash ipa/test/run_population_campaign.sh
# una famiglia aggiunta in fondo (strati): si misurano solo i modelli nuovi
CAMPAGNA=results/popolazione_2026-10 POP_T2=0 bash ipa/test/run_population_campaign.sh
    genera corretto pop solo
# rifare solo cio' che il report ha lasciato fuori
CAMPAGNA=results/popolazione_2026-10 POP_RIFAI_DISTURBATE=1 bash ipa/test/
    run_population_campaign.sh
# la sintesi e la figura di questo capitolo
python3 ipa/test/report_population.py results/popolazione_2026-10 > sintesi.md
python3 ipa/test/report_population_facts.py results/popolazione_2026-10 --figura
```

CAPITOLO 13

Cambiare modello senza fermare i pacchetti

13.1 La domanda

Un nodo in servizio riceve un modello nuovo. Quanto costa installarlo lo dice la sezione 12.12. Qui la domanda è un'altra: **che cosa vedono i pacchetti che arrivano mentre il modello cambia?** Per P1 e P1.5 la risposta è semplice: il modello sta dentro il programma, e cambiarlo vuol dire sostituire il programma intero, cosa che il kernel fa in un colpo solo. Per P2 e P3 il modello sta in più tabelle, e la risposta dipende da come le si scrive.

13.2 Le tabelle di un modello

In P2 e P3 un modello è fatto di cinque cose, scritte dal piano di controllo:

Tabella	che cosa contiene
pesi	un solo blocco condiviso da tutti i modelli, ciascuno nella sua zona (1 024 posti in P2, 2 048 in P3)
forme	in P3, una riga per strato: neuroni d'ingresso, d'uscita, dove stanno i pesi
descrizione degli ingressi	quali feature, di che larghezza, in che ordine
semantica delle classi	che cosa vuol dire ogni classe; due banchi per modello
registro	"questo `model_id` esiste", con la sua forma e il banco in uso

Il pacchetto porta nell'intestazione il `model_id`; il programma cerca quell'id nel registro e da lì legge tutto il resto. In P3 ogni programma della catena (il dispatcher, il primo strato, gli strati successivi) rilegge il registro.

13.3 Il problema

Il modo diretto di cambiare il modello A con il modello B è riscrivere le tabelle di A con i dati di B, **allo stesso** `model_id`, una tabella alla volta: i pesi, poi le forme, poi la semantica, poi il registro. Solo la semantica è protetta: si scrive nel banco libero, e la scrittura del registro scambia i banchi in un colpo solo. Tutto il resto cambia sotto i piedi dei pacchetti.

Un esempio

A ha 17 ingressi, 2 neuroni e 3 classi; B ha gli stessi ingressi, 8 neuroni e 7 classi. Un pacchetto che arriva quando i pesi sono già di B ma la forma dice ancora "2 neuroni" legge i pesi di B con la disposizione di A, e calcola un numero che non è né A né B. In P3 può anche cominciare la catena con il registro di A e finirla con quello di B. Se i dati che trova non stanno insieme, il programma rinuncia e passa il pacchetto al kernel senza decidere; se stanno insieme per caso, decide con un modello che non esiste, e nessun controllo se ne accorge.

13.4 La soluzione

Il programma eBPF non c'entra: conta il modo in cui il piano di controllo scrive le tabelle (`ipa/test/pipeline_setup.py`, `ipa/ebpf_template_arch.py`, `ipa/ebpf_modular.py`).

1. **Il registro per ultimo.** In P2 e in P3 pesi, forme, semantica e descrizione degli ingressi si scrivono prima del registro, che è l'ultima scrittura: finché non c'è, il modello non esiste; quando c'è, tutto il resto è già al suo posto.
2. **Mai riscrivere un modello in uso** (*make-before-break*):
   - `install_model` mette B sotto un `model_id` nuovo, in una zona libera del blocco dei pesi. A continua a servire il suo id; nessun pacchetto usa ancora B;
   - i mittenti passano al nuovo id. Ogni pacchetto porta il suo `model_id`, quindi usa tutto A o tutto B;
   - `retire_model` cancella l'id di A dal registro, una tabella hash: la cancellazione è una sola operazione, atomica. Un pacchetto che porta ancora l'id ritirato non viene elaborato (passa al kernel), mai calcolato con mezzo modello.

Scrivere B nel blocco dei pesi riscrive l'intero blocco, quindi anche la zona di A: ma con gli stessi byte, e chi legge A in quell'istante legge sempre A.

> Un `model_id` non cambia mai contenuto mentre i pacchetti lo usano: si installa il nuovo accanto al vecchio, si sposta il traffico, si toglie il vecchio. Ogni pacchetto vede un modello intero.

I limiti. A e B devono stare insieme nel blocco dei pesi (1 024 posti in P2, 2 048 in P3): il modello nuovo può occupare al più quello che il vecchio lascia libero. Per i modelli più grandi `install_model` si rifiuta, e il cambio richiede una breve interruzione. In P2 il modello nuovo deve avere anche la stessa profondità del programma compilato: una profondità diversa richiede un altro programma, non solo le tabelle.

13.5 La prova

`ipa/test/check_model_swap.py` cambia modello migliaia di volte mentre il programma lavora senza sosta. Un thread, su un core, esegue il programma un pacchetto alla volta (`BPF_PROG_TEST_RUN`, con gli ingressi del traffico); un altro, su un altro core, cambia il modello per 3 secondi. I modelli sono due della popolazione con gli stessi ingressi e larghezze diverse: per lo stesso pacchetto A decide la classe 1, B la classe 0. Alla fine si contano i pacchetti decisi con un'altra classe, o non elaborati mentre il loro id era registrato: sono quelli che hanno visto un modello misto.

	modo	cambi	pacchetti	pacchetti misti
P2	riscrittura allo stesso id	11 025	175 101	0
P2	make-before-break	3 311	569 829	**0**
P3	riscrittura allo stesso id	10 156	183 148	**60** (non elaborati)
P3	make-before-break	3 147	549 402	**0**

- **In P3 la riscrittura allo stesso id ha prodotto 60 pacchetti non elaborati**: arrivati a metà riscrittura, hanno trovato tabelle che non stavano insieme, e il programma li ha passati al kernel senza decidere. Su un nodo vero non avrebbero seguito la decisione della rete. È

lo 0,03 % dei pacchetti, con un cambio ogni 0,3 ms: su un nodo vero i cambi sono rari, ma il rischio c'è a ogni cambio. E la prova scrive già il registro per ultimo: senza, la finestra sarebbe più larga.

- **In P2 la riscrittura allo stesso id non ha mostrato pacchetti misti**, ma questo non la rende sicura: la finestra fra i pesi e il registro esiste, ed è solo più stretta; e un modello misto può dare per caso la classe giusta.
- **Con make-before-break zero pacchetti misti in P2 e in P3**, su più di un milione di pacchetti e 6 458 cambi in tutto. Nessun pacchetto ha trovato nemmeno un id appena ritirato: i mittenti passano al nuovo id prima che il vecchio sparisca.

I test del kernel che registrano i modelli passano con questo ordine delle scritture: `test_model_source -kernel` 110/110, `check_semantics_per_model` 53/53.

13.6 Quanto costa installare un modello

Registrare lo stesso modello tre volte di fila, in un processo nuovo, sullo stesso programma: la prima costa 0,62 ms in P2 e 1,11 ms in P3, la seconda e la terza circa 0,2 ms. La prima paga il piano di controllo che si prepara (gli oggetti delle tabelle, i moduli Python); la scrittura nel kernel è la stessa.

Il tempo di `bench_scaling`

Il banco degli assi (`bench_scaling`) misura un tempo più lungo per installare un modello in P2 e P3. Quel banco chiama la registrazione senza passarle il significato delle classi, e la funzione lo ricava leggendo i file del modello dentro il tempo misurato: quel tempo è quasi tutto lettura di file, non scrittura nel kernel.

13.7 Che cosa non dice

- **Il traffico della prova è il programma eseguito dal kernel su un altro core**, non pacchetti veri sul cavo virtuale: è la stessa concorrenza fra chi legge le tabelle e chi le scrive, senza la rete attorno.
- **P1 e P1.5 non sono nella prova**: cambiano modello sostituendo il programma intero, e il kernel garantisce che un pacchetto ne esegua uno o l'altro.
- **I modelli che non stanno due volte nel blocco dei pesi** richiedono un'interruzione.

13.8 Come rifarla

```
sudo python3 ipa/test/check_model_swap.py --out results/popolazione_2026-10/cambio_modello
sudo python3 ipa/test/test_model_source.py --kernel
sudo python3 ipa/test/check_semantics_per_model.py
```

CAPITOLO 14

I numeri in una pagina

	baseline	P1 statica	P1.5	P2	P3
Il nodo su traffico vero, un P-core (capitolo 9), 64 byte					
milioni di pacchetti al secondo	15,0	8,4	8,0	3,4	2,1
ns di CPU per pacchetto	66	119	124	292	487
costo della rete (sopra la baseline)	—	53 ns	58 ns	226 ns	421 ns
istruzioni eseguite per pacchetto	606	1 168	1 349	4 578	7 620
istruzioni per ciclo	2,6	2,9	3,2	4,5	4,5
Lo stesso nodo su un E-core e su un LP E-core					
milioni di pacchetti al secondo, E-core	9,4	6,4	6,0	2,8	1,7
rispetto al P-core	0,63	0,76	0,75	0,82	0,83
LP E-core rispetto al P-core	0,26	0,35	0,35	0,45	0,48
Il solo programma					
$T_2 - T_1$ sul percorso vero (minimo)	27 ns	64 ns	70 ns	229 ns	391 ns
la rete da sola, in un ciclo caldo	—	42 ns	48 ns	214 ns	381 ns
istruzioni del programma caricato	135	650	961	16 813	12 288
letture di tabella per pacchetto	3	5	6	12	36
Cambiare modello su un nodo in servizio					
caricare sul nodo	—	0,6 ms	0,9 ms	0,3 ms	0,3 ms
anche generare e compilare	—	50 ms	61 ms	non serve	
serve un compilatore sul nodo?	—	no (compilato prima)		no	

Mediane sulla popolazione (`results/popolazione_2026-10/`): 109 modelli sul P-core (P2: 103), 108 sull'E-core, 14 sul LP E-core, 91 con i tre orologi; core del banco a 3,5 GHz (2,5 il LP E-core).

In quattro righe:

1. **Scrivere i pesi nel programma costa in mediana 50–60 ns per pacchetto sul nodo; leggerli da una tabella 225–420 ns.** È il prezzo della configurabilità, e si stima dalla forma del modello con la legge di costo, con qualche punto percentuale di errore.
2. **Con l'oggetto precompilato, cambiare modello costa meno di un millisecondo in mediana per tutte le pipeline**. Il vantaggio residuo di P2 e P3 è non dover compilare niente, da nessuna parte.
3. **Congelare anche il nodo (P1 statica) vale poco ma si vede**: la rete di P1.5 costa 1,11 volte quella di P1 in mediana, di più su 89 modelli su 109, e la dimensione di P1 non dipende più dalla taglia della rete.
4. **Su un core lento il codice è lo stesso, l'IPC no**: un E-core regge dal 63% (baseline) all'82–83% (P2, P3) del P-core, perché rallenta soprattutto la parte fissa del nodo e poco l'aritmetica della rete.

CAPITOLO 15

Quel che resta aperto

- **Il costo del nodo di baseline e** `rxonly` con pacchetti grezzi: con un thread per coda il generatore non le satura sul P-core, e più thread intralciano la coda (sezione 9.8). Serve un generatore più veloce su un solo scrittore, o una scheda vera.
- **Il ritardo end-to-end sulla popolazione**: le curve a traffico crescente lo registrano, ma la sintesi non lo riassume.
- **Altre frequenze**: tutte le misure sono a 3,5 GHz (2,5 il LP E-core).
- **Gli E-core al loro massimo**, 3,8 GHz: misurati solo a 3,5.
- **Hardware vero**: una scheda di rete con XDP nativo e una seconda macchina, per un numero che non sia di un cavo virtuale.
- **Il tetto sull'uscita di P2**: lo strato finale è srotolato a 32 classi per un modello che ne ha 7. Abbassarlo ridurrebbe il programma ma *restringe l'insieme dei modelli rappresentabili*.
- **I salti fra programmi di P3**: uno per strato, da 2 a 7 per pacchetto sulla popolazione, ed è lì (con le letture che li accompagnano, circa 690 istruzioni fisse per strato) che sta la latenza. Quanto sia il salto e quanto le letture non è separato.
- **La profondità massima di P2** fra 20 e 37 strati: il punto esatto in cui clang si ferma non è misurato.
- **Isolamento dal boot**: i core del banco sono isolati a sistema acceso; l'isolamento completo (tick e callback del kernel) richiede parametri di avvio.

CAPITOLO 16

I comandi

Tutti i comandi si lanciano dalla radice del repository. `sudo` serve ovunque si carichi un programma nel kernel o si tocchi un'interfaccia; senza `sudo` girano solo le prove che non escono da Python.

Prerequisiti

Linux con BCC installato; per l'oggetto AOT di P1 anche

```
sudo apt install clang libbpf-dev libelf-dev zlib1g-dev libzstd-dev liblzma-dev
```

Il generatore (`xdp_gen`) è uno script del repository; il modulo `pktgen`, per l'alternativa, il banco lo carica da sé.

16.1 La macchina

```
# topologia, piano dei ruoli e condizioni attuali (niente root)
python3 ipa/test/host_conditions.py --show

# un comando qualunque sul core del nodo, a condizioni applicate
sudo python3 ipa/test/host_conditions.py --run -- CMD ...

# se un run e' stato ucciso: rimette la macchina com'era
sudo python3 ipa/test/host_conditions.py --restore

# le condizioni verificate: su un /sys finto, e sul kernel vero
python3 ipa/test/test_host_conditions.py
sudo python3 ipa/test/test_host_conditions_kernel.py --bench

# tutte le misure senza traffico vero (circa 20 minuti)
bash ipa/test/run_kernel_checks.sh
# la campagna sulla popolazione
bash ipa/test/run_population_campaign.sh
```

16.2 Correttezza: il modello fa quello che deve

```
python3 ipa/test/test_weights_and_metrics.py
python3 ipa/test/test_class_semantics.py
python3 ipa/test/test_synthetic_models.py
python3 ipa/test/check_kernel_vs_reference.py --all --dry-run
```

16.3 Kernel: si carica, e quanto costa

```
sudo python3 ipa/test/host_conditions.py --run -- \
     python3 ipa/test/test_weights_and_metrics.py --only kernel
sudo python3 ipa/test/check_multi_model.py
```

```
sudo python3 ipa/test/check_semantics_per_model.py
sudo python3 ipa/test/check_kernel_vs_reference.py --all --n 300
sudo python3 ipa/test/check_kernel_vs_reference.py --all --n 300 --pipeline p2
sudo python3 ipa/test/check_kernel_vs_reference.py --all --n 300 --pipeline p3
```

16.4 Altri modelli, altre reti

```
python3 ipa/test/test_model_source.py
sudo python3 ipa/test/test_model_source.py --kernel
sudo python3 ipa/test/host_conditions.py --run -- \
     python3 ipa/test/test_weights_and_metrics.py --only kernel --model synth:deep
sudo python3 ipa/test/test_forwarding.py --model synth:small -q
```

16.5 Datapath reale: il pacchetto attraversa davvero

```
sudo python3 ipa/test/test_forwarding.py
sudo python3 ipa/test/test_forwarding.py --method aot
sudo python3 ipa/test/test_forwarding.py --sweep
sudo python3 ipa/test/netns_fabric.py --cleanup
```

16.6 Analisi parametrica: come cresce il costo

```
sudo python3 ipa/test/host_conditions.py --run -- \
     python3 ipa/test/bench_scaling.py --axis all --out results/
sudo python3 ipa/test/host_conditions.py --run -- \
     python3 ipa/test/bench_scaling.py --axis campaign --out results/
python3 ipa/test/bench_scaling.py --plot results/
sudo python3 ipa/test/bench_scaling.py --verify
sudo python3 ipa/test/bench_scaling.py --verify-ports
sudo python3 ipa/test/host_conditions.py --run -- \
     python3 ipa/test/bench_depth_vs_width.py
```

16.7 Il test reale

```
# tutto il traffico sulla popolazione, poi la sintesi
bash ipa/test/run_population_campaign.sh
python3 ipa/test/report_population.py results/popolazione_2026-10 > sintesi.md

# capacita' a un core: nodo su un P-core (6) o su un E-core (12)
sudo python3 ipa/test/bench_capacity.py --mode compare --generator xdp \
     --rounds 3 --gen-cpus 10 --dut-cpus 6 --egress-cpu 8 --out results/pcore
sudo python3 ipa/test/bench_capacity.py --mode compare --generator xdp \
     --rounds 3 --gen-cpus 10 --dut-cpus 12 --egress-cpu 8 --out results/ecore

# il bit rate crescente: latenza, ricevuti, inoltrati, dove si perde
sudo python3 ipa/test/bench_load_curve.py --generator xdp --gen-cpus 10 \
     --dut-cpus 6 --egress-cpu 8 --out results/bitrate
python3 ipa/test/bench_load_curve.py --report results/bitrate

# le tre marcature T1/T2/T3 su una scala di rate
sudo python3 ipa/test/bench_capacity.py --mode rates --generator xdp \
```

```
     --gen-cpus 10 --dut-cpus 6 --frames 64 --rounds 3 --rates 0.05,0.5,1,1.5,2,2.5,3

# per classe e per taglia
sudo python3 ipa/test/bench_capacity.py --mode compare --generator xdp \
     --gen-cpus 10 --dut-cpus 6 --egress-cpu 8 --per-class --rounds 3
sudo python3 ipa/test/bench_capacity.py --mode compare --generator xdp \
     --gen-cpus 10 --dut-cpus 6 --egress-cpu 8 --frames 64,512,1514 --rounds 3

# i contatori hardware, a mano
sudo python3 ipa/test/hw_counters.py --cpu 6,12,20 --seconds 1

# la finestra di misura e le formule, verificate senza kernel
python3 ipa/test/test_capacity_window.py
python3 ipa/test/test_load_curve_math.py

# se un run si interrompe e lascia il generatore configurato
sudo python3 ipa/test/bench_capacity.py --cleanup
```

Come si legge l'uscita

Condizioni del test — in testa, e in `env.csv`: ruoli, frequenza scritta e misurata, temperatura, interventi di throttling durante il run.
Controllo di validità — la baseline deve essere la più veloce entro il 10% e `rxonly` non deve essere più lento della baseline. Se una delle due cose non vale, il run misura il banco e non le pipeline.
Finestre marcate — «macchina» (fuori dalle condizioni dichiarate), «raffica» o «gen» (il generatore non ha offerto il carico chiesto): contate e riportate, non scartate.

16.8 Costi specifici e oggetto precompilato

```
sudo python3 ipa/test/bench_tail_call.py
python3 ipa/methods/method4_hardcoded_aot.py --build-only   # build, come utente
sudo python3 ipa/methods/method4_hardcoded_aot.py           # verifica e bench
sudo python3 ipa/test/check_verifier_stats.py                      # statistiche del
    verificatore
```

16.9 L'ordine consigliato

Partendo da zero su una macchina nuova:

1. `python3 ipa/test/host_conditions.py -show` — com'è fatta la macchina, e dove andrebbe il banco?
2. `python3 ipa/test/test_synthetic_models.py` — il C generato per P1 calcola il modello? (`test_weights_and_metrics.py`, che controlla estrazione dei pesi e quantizzazione, richiede PyTorch)
3. `sudo python3 ipa/test/test_host_conditions_kernel.py` — le condizioni si mettono e si tolgono davvero?
4. `sudo python3 ipa/test/test_forwarding.py` — il pacchetto attraversa davvero?
5. `bash ipa/test/run_kernel_checks.sh` — tutte le verifiche nel kernel.
6. `bash ipa/test/run_population_campaign.sh` — tutti i numeri.